Early Detection of Transmission Gaps in Remote Patient Monitoring

A Capstone Project Report of MSIT 5910

Submitted by:

Anam Kazani

For the partial fulfilment of the requirements for the degree of

Master of Science in Information Technology

Supervised by:

Dr. Sirisha

Department of CS & MSIT

University of the People, Pasadena, CA 91101, United States

Fall Term, October 2026

ABSTRACT

Remote patient monitoring programs depend on patients continuing to transmit readings from home medical devices, yet a substantial minority stop transmitting during enrollment, and that silence is typically discovered weeks later through end-of-cycle reporting. This project designed and built a transmission-gap detection layer that converts data a monitoring program already holds into an early, ranked, and explainable outreach queue. The literature establishes that remote monitoring produces measurable clinical benefit, that attrition concentrates among patients with poorer baseline health, and that alerting systems ignoring patient-specific context degrade into noise clinicians learn to disregard. No existing work addresses the operational middle ground of converting transmission logs into a timely work signal. The system uses a layered architecture of four modules: an ingestion service that reduces vendor records to three fields before persistence, a scoring engine that measures silence against each patient's own rolling thirty-day baseline, a queue API enforcing server-side authorization, and a disposition logging module that captures outreach outcomes and supports cohort fairness measurement. The design embeds data minimization, consent filtering, and per-patient thresholds as specified requirements rather than implementation conventions. On 240 synthetic patients, the personal baseline flagged all 64 devices that needed a call with no false alarms, while a fixed 48-hour rule raised 40 false alarms and missed eight. System testing with 46 test cases found four defects, all fixed in version 0.4.0. The project demonstrates a detection mechanism using synthetic data and makes no clinical outcome claims.

ACKNOWLEDGEMENT

I would like to thank my instructor, Dr. Sirisha, for guidance throughout this capstone course, the department chair and the Department of Computer Science and MSIT for the structure of the program, and the University of the People for making graduate study in information technology accessible.

LIST OF TABLES

Table 1. Versions and Milestones — 28

Table 2. Web API of the Integrated System — 33

Table 3. Test Summary Report — 43

Table 4. Defects Found and Fixed — 44

Table 5. Accuracy and Detection Time on 240 Synthetic Patients — 46

Table 6. Load Test Results — 48

Table 7. Interface Clarity Walkthrough — 49

Table 8. Non-Functional Requirements Against the Evidence — 53

Table B1. Sample Rows From the Synthetic Vendor File — 67

Table B2. Settings and Their Defaults — 67

LIST OF FIGURES

Figure 1. System Architecture for Transmission-Gap Detection — 22

Figure 2. Main Logic of Modules 1 and 2 — 31

Figure 3. Main Logic of Modules 3 and 4 — 32

Figure 4. Ingestion and Scoring Output — 35

Figure 5. API Requests and Replies — 35

Figure 6. Nurse Dashboard After Sign-In — 36

Figure 7. Dashboard-to-API Link — 36

Figure 8. API, Module, and Database Links — 37

Figure 9. Unit Tests Passing Against Version 0.3.0 — 38

Figure 10. First Test Run Against Version 0.3.0 — 40

Figure 11. Why the Four Tests Failed — 40

Figure 12. Fix for Defects D2 and D4 in log_outcome() — 41

Figure 13. Second Test Run After the Fixes (Version 0.4.0) — 42

Figure 14. Performance Test Output — 42

Figure 15. Evaluation Output — 46

Figure 16. Precision, Recall, and F1 Score of the Two Rules — 47

Figure 17. Hours From Last Reading to Flag, by Sending Rhythm — 48

Figure 18. Dashboard Message for a Missing Outcome — 50

Figure A1. Fix for Defect D3 in clean_time() — 65

Figure A2. Core Scoring Functions usual_gap() and score_device() (Version 0.4.0) — 65

Figure C1. Dashboard After a Call Is Logged — 69

SYMBOLS AND ABBREVIATIONS

API: Application programming interface

AT, IT, PT, RT, ST: Acceptance, integration, performance, regression, and system test IDs

CI/CD: Continuous integration and continuous delivery

CSV: Comma-separated values

FR, NFR: Functional requirement, non-functional requirement

HTTP, HTTPS: Hypertext Transfer Protocol, and its secure form

JSON: JavaScript Object Notation

MFA: Multi-factor authentication

REST: Representational state transfer

RPM: Remote patient monitoring

RQ: Research question

TLS: Transport Layer Security

UTC: Coordinated Universal Time

h, ms, s: Hours, milliseconds, seconds

Score: Hours of silence divided by the device's usual gap

CONTENTS

Abstract — 2

Acknowledgement — 3

List of Tables — 4

List of Figures — 5

Symbols and Abbreviations — 6

INTRODUCTION — 10

1.1 Background of the Study — 10

1.2 Problem Statement — 10

1.3 Objectives of the Study — 12

1.4 Scope and Limitations — 12

1.5 Report Organization — 14

LITERATURE REVIEW — 15

2.1 Remote Patient Monitoring and the Problem of Attrition — 15

2.2 Engagement Frequency as a Meaningful Signal — 16

2.3 Workflow Friction and the Risk of Alert Fatigue — 17

2.4 Implementation Barriers in Practice — 18

2.5 Methodological Grounding — 18

2.6 The Identified Gap — 18

SYSTEM DESIGN AND METHODOLOGY — 20

3.1 Architecture and Its Connection to the Problem — 20

3.2 Module Specifications — 23

3.3 Functional Requirements — 24

3.4 Non-Functional Requirements — 24

3.5 Data Collection Approach — 25

3.6 Tools and Version Control — 26

RESULTS AND EVALUATION — 27

4.1 Milestones Achieved — 27

4.2 Development Across Units 4 to 7 — 28

4.3 Core Algorithm Implementation — 29

4.4 System Integration Summary — 32

4.5 Verification of the Integrated System — 37

4.6 System Testing — 39

4.7 Evaluation Metrics and Results — 45

DISCUSSION — 50

5.1 Analyzing the Findings in Light of Prior Research — 50

5.2 Deployment Approach and Configuration Summary — 53

5.3 Limitations — 55

5.4 Recommendations for Future Research — 56

CONCLUSION AND FUTURE WORK — 56

6.1 Summary of Main Findings and Implications — 56

6.2 Contributions to the Field — 57

6.3 Lessons Learned — 57

6.4 Challenges — 58

6.5 Next Improvements — 59

References — 61

Appendix A: Code Samples — 65

Appendix B: Data Samples and Settings — 67

Appendix C: Additional Screenshots — 69

INTRODUCTION

1.1 Background of the Study

Remote patient monitoring is the blend of health care and health information technology. Patients enter the program and are sent home with monitoring tools such as a blood pressure cuff, an oximeter, or a weighing scale. The devices transmit data to a central database where nurses assess the values. The overall success of the program depends on continuity of data flow from patients. If a device does not send readings anymore, the reasons may vary from a dead battery and an unplugged cellular hub to the admission of patient to hospital or patient stopped participating in the program.

In most programs that silence is noticed late. A monthly report surfaces the patient only after they have already fallen below the transmission threshold the program depends on, which means weeks have passed with no monitoring at all. The problem belongs to the data analytics and workflow automation side of health IT rather than to device engineering, because the data needed to catch the gap already sits in the platform's transmission logs. Nothing converts that data into a timely, prioritized work queue.

1.2 Problem Statement

Current remote patient monitoring programs identify transmission failures of patients too late in the cycle, usually at the end of the billing period due to insufficient manipulation of transmission data into early warning signals for outreach. The proposed work involves formulating and implementing a flagging process and dashboard powered by a set of rules that enable

identification of patients missing transmission signals within a set period of time and assigning the cases to the relevant follow-up steps.

Problem Context

The concept of the ideal state is not complex. A patient does not send information, and a few days later the software detects this fact, a person calls the patient, finds out what is the matter, and the issue is resolved before too much time passes without monitoring. In reality, situations work differently. Issues are raised when scheduled reviews happen at the end of the billing cycle, and after weeks of monitoring has already been missed. Employees do not have any idea which patients are quiet and dangerous, so they divide their efforts evenly between all patients. This has negative consequences for patients and clinic.

Stakeholders

Patients are the ones affected the most because the gap in information from patients leads to the fact that the worsening of the patient's condition remains unobserved. Su et al. (2025) elaborate on this issue by saying that because dropout rates are higher among patients with poor initial indicators, quiet patients cohort is likely to include patients in poorer health who cannot afford to lose contact with the doctors.

The monitoring nurses are the next ones suffering from the situation. Due to absence of queues of patients, they have to either try to address the issues with all patients or wait for the monthly report. The program managers suffer from problems with the number of drops in the program of information processing. The technical support department suffers as problems with the devices and inactivity of the patients are mixed and need to be separated through a phone call rather than

with the help of data. Resolving the issue allows re-establishing continuous monitoring and working with patients with lesser number of calls.

1.3 Objectives of the Study

The project includes four goals. The first goal is to create a set of rules and a triage process that sorts transmission problems into categories that can be acted upon, rather than treating all silence the same. The second goal is to create a scoring system that sorts patients who did not transmit messages using deviations from their transmission patterns. The third goal is to make it possible to view the queue of patients in a user-friendly manner. Lastly, the fourth goal is to analyze the outcome using quantitative observation measures and a usability review of the system.

Three questions will be considered. First, does the comparison of an individual patient's transmission patterns with a program-wide day threshold decrease the number of false positives? Second, does the newly formatted queue reduce the delay in detecting silent devices compared with end-of-cycle reporting? Third, do flag rates differ across patient groups in ways that reflect connection problems rather than genuine disengagement?

1.4 Scope and Limitations

The project comprises the following elements: a summary of the requirements defining the current workflow; a data model that relates to the logs of the system; a set of rules that categorizes a non-operational device as a probable technical failure/disengagement; a prototype of a dashboard that displays the resulting listing by priority; a documentation of the workflow of classifying; a walkthrough of usability; testing of the system; and a summary report about the project.

Exclusions from the work are as follows: no integration with a current EHR or platform used in production; no patient data; no claims regarding clinical outcomes; no machine learning; and no deployment of the result. The work requires using synthesised data that will be realistic enough to apply the rules. According to Oguz (2025), every project operates under the control of a competing constraint of time, costs, and scope. Since the time schedule is fixed in the term length of the work, and budget is effectively equal to zero, one person provides all the workload. As the timeline and resources are fixed, scope has to absorb every problem encountered by the project. As a result, exclusions are made clear at the start rather than negotiated later.

Ethical Considerations

Privacy comes first. Transmission logs constitute patient data even without vital signs attached, because the pattern of when someone uses a home medical device is itself revealing. Using real logs would require consent and institutional approval a student project cannot reasonably obtain within a term, which makes synthetic data a design decision rather than a shortcut.

Fairness is the second concern and is specific to this design. A rule that flags patients purely by days of silence will disproportionately flag people with unreliable broadband, older patients less comfortable with the equipment, and rural households. Read as a list of disengaged patients, such a queue would encode a connectivity problem as a compliance problem. The rules must therefore separate suspected technical causes from behavioral ones and must never present a flag as a judgment about the patient.

Intellectual property is a third consideration, since workflow knowledge informing this project comes from a professional setting; the project describes processes in general terms and uses only openly licensed tools. A fourth is honesty in estimation. Oguz (2025) treats cost and risk

estimation as producing commitments others rely on, so a proposal that understates effort to appear feasible misleads its reader.

Security Implications

Even the prototype can expose vulnerabilities: a dashboard on an accessible port, a repository of configurations with real service names, or an old version of a visualization library. Access management is crucial since an unrestricted dashboard that contains actual data would allow the full patient roster to become vulnerable. According to Oguz (2025), risk management is about identifying risks, assessing probability and impact, and selecting the respective response. Therefore, the tools discussed here should rather be included into the risk register than represented in a separate document.

The primary mitigation is avoidance, which Oguz (2025) lists among the available risk responses: using only synthetic data removes the highest-impact risk instead of managing it. Supporting controls follow. Files stay in a repository with no patient-derived content. The prototype binds to localhost and requires authentication before any demonstration. The prototype uses only Python's standard library, so there are no third-party dependencies to patch. Role separation is modeled in the design so that moving to real data would require adding credentials rather than removing a shortcut.

1.5 Report Organization

According to Miller (2024), the capstone report goes through the literature and method in order, and after that moves to results, discussion, and the conclusion section. Thus, this report,

following that order, is reflected in Chapters 2-6, and the appendix contains code samples, data samples and settings, and an additional image.

LITERATURE REVIEW

2.1 Remote Patient Monitoring and the Problem of Attrition

As a new technology, remote patient monitoring has evolved into an essential part of chronic disease treatment. However, when surveying the literature, researchers point to continuation of patient participation as the obstacle to the success of the program rather than the effectiveness of the devices or procedures. According to Athukorala (2026), large deployments of remote patient monitoring programs fail due to low compliance of patients and unwillingness of the providers to implement these solutions. The review also confirms that the longevity of the remote patient monitoring program deployment depends on making appropriate choices regarding technical architecture and the optimization of the subsequent physician workflows. That idea forms the basis for this paper, which suggests that the issue does not lie in the devices used but in the workflows applied.

Su et al. (2025) provide the most direct evidence of the existence of the attrition problem. They conduct a mixed-methods study of the experience of a Nebraska program which serves as the relevant empirical touchstone for the present research. Of 1,993 patients with diabetes invited to participate, 13% declined, 16% withdrew before completion, and 71% finished the program. The logistic regression analysis indicates that age and health condition (high level of glucose or blood

pressure) were significant predictors of attrition. The main reason why the patients opted against participation or withdrew from the program was inability to cope with the time needed to finish it.

Thus, there are two implications of this research for the present project. First, around one in six patients leave the program before finishing it, thus indicating the importance of the need to respond to the case of transmission silence in a systematic way rather than rely on a chance follow-up. Second, and far more importantly, the drop-off amongst patients is among those with the worst baseline measures, which means that the patients that are likely to be lost are the ones that most require monitoring. Hence, a detection system that identifies problems late only fails most where a failure is most detrimental. This is an important conclusion that redefines the project from being an efficiency improvement initiative into a patient safety initiative and serves as the most important reason in the literature to detect lapses in days rather than weeks.

2.2 Engagement Frequency as a Meaningful Signal

Apart from this, it is supported by the literature to regard transmission frequency as having medical value rather than being an administrative indicator. Michaud et al. (2018) have assessed medical outcomes for patients who left their hospital after being diagnosed with type 2 diabetes and participated in remote monitoring, providing an argument for the outcome risk that is threatened by attrition. Athukorala (2026), referring to the side of the program implementation, supports this conclusion, pointing to the longevity of the program being dependent on the constant level of engagement, not on the technical aspects of the monitoring equipment.

The relevance of this argument for the architecture outlined later in the report is that it adds justification to the approach of being able to score the cases of trouble in the functioning of the

program. If the frequency of transmission had no impact on the efficiency of the program, then a queue based on the combination of cases of silence would only be a matter of convenience for operational purposes, and that would be it because the effectiveness of the remote monitoring program comes from the fact that it is itself ongoing in time. A lapse in transmission is therefore a weak but real warning sign, and detecting it earlier carries value.

2.3 Workflow Friction and the Risk of Alert Fatigue

The second group of studies warns against the most obvious solution, which was to create an alert to log any occurrence. Ancker et al. (2017) showed that in clinical decision support, the problem of alert fatigue arises due to workload, complexity, and the number of repeated alerts, which means that as the total number of notifications clinicians tend to respond to every signal less and less. This means that a highly prone system not only wastes the time of users, but it also forces them to ignore alerts.

This gives justification for the two particular design solutions explained in Chapter 3. The per-patient rolling baseline rather than the program threshold is a clear response to this risk, as a patient with an unstable cadence will not cause an alert just because he/she acts normally. Furthermore, the fact that every score has to come with an obvious explanation makes sure that alerts that a nurse has to evaluate before taking action do build trust rather than destroy it.

Athukorala (2026) confirms this problem from the patient perspective, reporting that technology anxiety and alert fatigue were present in 9.3% of the sample and gaps in digital health literacy were shown in 15%, while workflow friction goes hand in hand with illiteracy problems. A

system that equates connectivity issues and genuine disengagement as identical would make the issue an infrastructure or literacy problem into one of apparent compliance, which is the fairness risk this study's structure makes sure to differentiate from.

2.4 Implementation Barriers in Practice

Kirkland et al. (2023) analyze the methods of delivery as well as the facilitators and obstacles in remote diabetes monitoring, as it provides an operational perspective to the previous outcome-oriented research above. Their study is framed as one related to implementation science, ignoring the technological aspect of remote patient monitoring issues, thus supporting the framing of the present capstone project as a workflow or process intervention rather than a project on the device or algorithm.

2.5 Methodological Grounding

Various sources have defined how the study is organized regardless of its content. Sheppard (2019) defines literature survey as a synthesis revealing how the recent study contributes to the existing research and acknowledges the gap the study is going to fill that is the organizational principle of the current paper. PhD Assistance (2020) gives the criteria for determining whether the research problem is logical, significant, and genuinely unresolved, which scale down to a one-term project. Paperpile (n.d.) suggests structuring the problem statement by stating the ideal state and comparing it to the real one, which is what is done in Section 1.2. Oguz (2025) says that most project difficulties arise at the point of initiating the project since the absence of clear objectives and undefined scope leads to problems already before the project begins.

2.6 The Identified Gap

In general, the literature provides insight into three questions yet leaves one unanswered. It proves that remote monitoring offers measurable clinical benefit, that a large number of patients drop out, and that alerting systems ignoring the patients' context become noise that clinicians get used to ignoring.

One question remains unanswered, which is how the monitoring program manages to transform its logs into early outreach signals prioritized by importance. Su et al. (2025) measured retention in a retrospective fashion when the patients had already quit the program. Athukorala (2026) pointed to the optimization of downstream workflow as the key factor for prolonging the existence of the program, yet did not clarify its content. Ancker et al. (2017) show the consequences that a poor signal has on clinicians but deal with clinical decision support instead of focusing on engagement monitoring.

The research tackles that gap by developing the detection layer that estimates the duration of transmission silence for every individual patient, generates an explanation for the score, and assesses the effectiveness of its work. It does not deal with forecasting clinical decline or demonstrating proven clinical outcomes, as those issues go beyond its one-term scope and require real patient data. The objective is much narrower and simpler, as the study is focused on establishing whether the available data can be effectively transformed into an actionable queue.

SYSTEM DESIGN AND METHODOLOGY

3.1 Architecture and Its Connection to the Problem

The layer architecture is made up of four distinct modules added between the vendor data source and the monitoring nurse. The four modules consist of the ingest service, the scoring engine and gap detection engine, the dashboard and queue layer, and finally, the logging module for disposition. Walker (2022) explains how the layer pattern segregates tasks in such a way that any failure or issue is contained within one of the four layers, thus benefiting the system in that it is able to keep privacy and authorization in line as the modules are made to function together.

Data flows through the different modules in one direction, from vendor records coming in and the queues going out, while in the case of the disposition module, the data goes back to the scoring engine and not the scoring path.

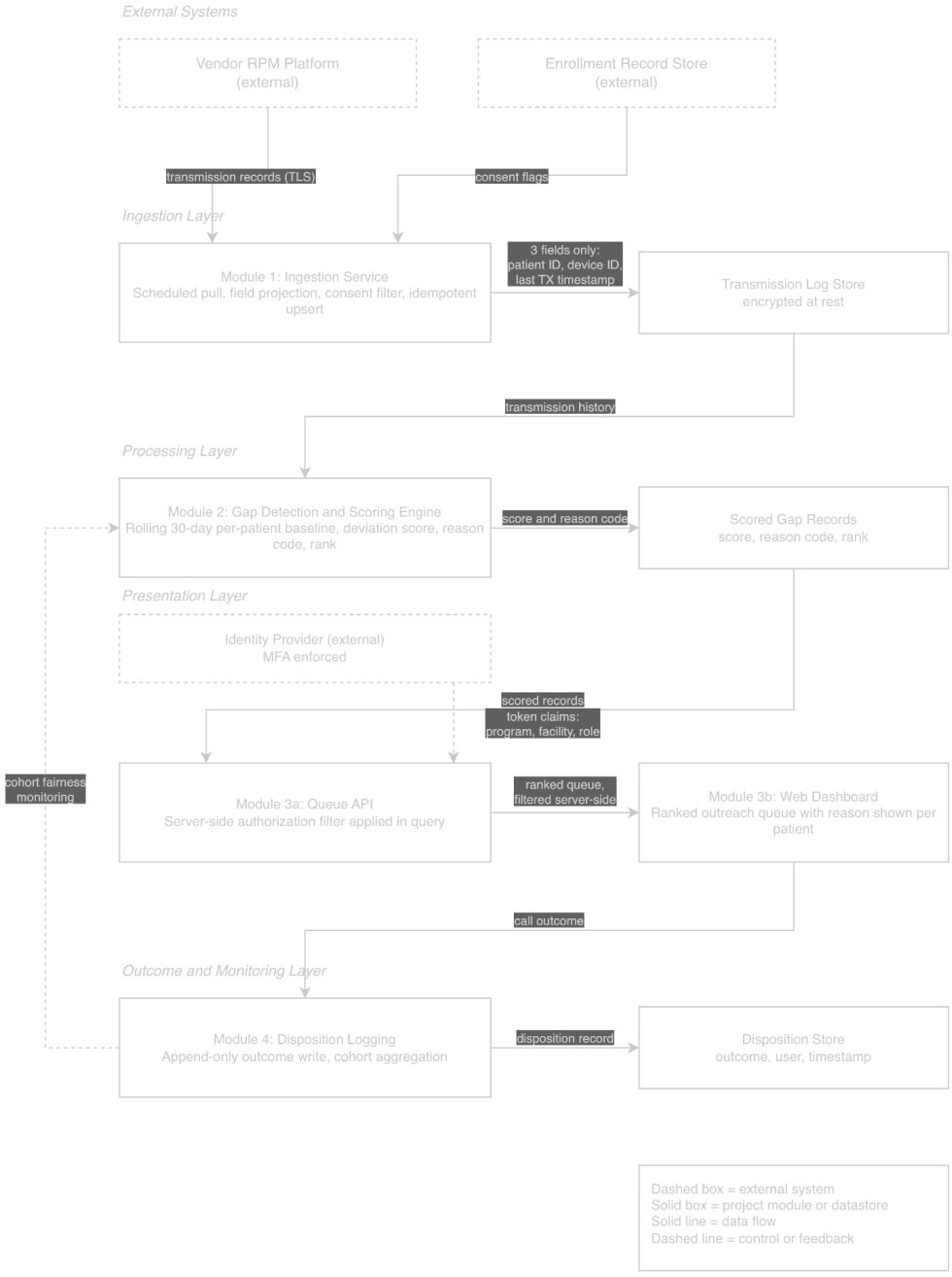
There are three points that can be tied to an issue under consideration through using the architecture. In particular, the ingest service has dealt with the issue of privacy that was mentioned in Section 1.4 by reducing the number of fields stored in the database to only three. The scoring engine is responsible for solving two issues at the same time: the one about latency and about fairness, since it comes up with values for each patient, while the disposition module deals with the issue of measurement gap by virtue of making the fairness of the cohort demonstrable information.

Figure 1 shows how all units interrelate: the vendor's RPM platform and the record store serve as outer sources; then there comes the ingest service utilizing both of them, the log database, the scoring engine applying the data from the log database, the dashboard and queue API

interpreting the logged data through the security filter, the identity provider checking people using the dashboard, and finally coming to the disposition store again.

Figure 1

System Architecture for Transmission-Gap Detection



3.2 Module Specifications

Inputs to the ingestion module consist of two components, the transmission records obtained from the vendor’s platform at scheduled intervals, along with enrollment records that show each patient’s monitoring and outreach consent status. The output of the process is a series of cleaned-up transmission events which contain the patient ID, device ID, and last-transmission time. The method applied is that of acquiring the transmission records in batches, which means that before anything is recorded, the record data are processed in such a way that only the above three fields will be available at the end. It should also be noted that the write procedure is idempotent so that even if the operation is repeated, history will not be repeated again. According to Summers (2020), it is vital to define the subsystem requirements clearly enough, which will help to make sure that the components of the system are working efficiently and in accordance with the overall goal, which is exactly what this projection conveys - it is a contract instead of a convention.

The delay analysis engine takes as input the transmitted event logs of the patient and device and delivers as output the gap score and ranking. The method used for the delay analysis is a rolling baseline of the last thirty days of transmission of each patient; in other words, the current silence is compared against the patient's own median gap in order to produce the gap score. Determining the threshold value for all patients collectively would be easy, but since this method would misinterpret patients who usually have irregular transmission patterns, the system keeps an individual log of each patient instead. Each score is associated with a gap explanation.

The queue and dashboard layer takes as input the gap records and an authenticated session key and delivers as output the ordering of the outreach queue in a web interface. The method here is

a REST API that checks the caller's program on the server, so authorization is enforced by the service returning the data, while authentication is left to the organization's identity provider.

The log processing module takes the outcome of the outreach process, written by the nurse, whether device failure, hospitalization, disengagement, unreachable, or resolved, and outputs an append-only record with a timestamp, which is later compared across patient groups to measure fairness.

3.3 Functional Requirements

FR-1: The system shall ingest transmission records from the vendor platform on a configurable schedule. FR-2: The system shall persist only the patient identifier, device identifier, and last-transmission timestamp from each record. FR-3: The system shall exclude any patient without an active monitoring and outreach consent flag. FR-4: The system shall compute a gap score for each enrolled patient relative to that patient's own thirty-day transmission baseline. FR-5: The system shall emit a reason code with every score. FR-6: The system shall present a ranked outreach queue filtered to the authenticated user's assigned programs. FR-7: The system shall record a disposition outcome for each queue item acted upon. FR-8: The system shall produce cohort-level flag-rate and confirmed-problem-rate reports.

3.4 Non-Functional Requirements

NFR-1 (security): Utilization of TLS will be mandatory for all inter-module traffic and data stored in the transmission log will be encrypted during storage. NFR-2 (security): Authorization will be carried out on the server-side with a negative test verifying that a user assigned to one program will have no results for any other program. NFR-3 (performance): The complete scoring execution will be finished within the time assigned for the execution. NFR-4 (usability): The

required explanation for the ranking of each patient will be available in the queue for the nurses to examine before calling. NFR-5 (reliability): Ingestion will be idempotent and able to recover from a failed run without manual reconciliation. NFR-6 (scalability): The scoring will be carried out separately from the dashboard to maintain the response speed of the interface despite the increase in the number of patients. NFR-7 (fairness): The discrepancies between the flagging rates for various cohorts will also be determined based on the data collected in the database.

Letaw (2024) observes that good requirements can be tested and that non-functional requirements such as security, performance, or usability should be a part of the specification together with behavioral requirements instead of being presumed only. In addition, Krüger (2025) points out that the purpose of requirements specification is to make all of the expectations explicit and verifiable before the code is written; this is why NFR-2 is stated in the form of a statement that can be assessed instead of a regular requirement.

3.5 Data Collection Approach

Because real transmission logs are excluded from scope, the system is exercised against a synthetic dataset of at least 200 simulated patients. The generator produces transmission histories with deliberately varied patterns: patients who transmit daily, patients who transmit reliably but only within a narrow time window, and patients on intermittent connectivity whose baseline gaps are naturally wider. Known gaps are seeded at defined points so detection accuracy can be measured against ground truth, which is what makes the quantitative evaluation in later units possible at all.

3.6 Tools and Version Control

The project is designed in a Git repository divided into four components: /src for code, /tests for both unit testing and integration, /docs for usage specifications and reports, and /design for the architecture diagram and interface contracts. Positioning documentation together with codes indicates that design and implementation systems are in sync; Editorial Team (n.d.) and Lucidchart (n.d.) also note that the real value of design documentation comes from the preservation of the justification behind the structural choice for people maintaining that system.

The branch model consists of two long-lasting branches. The base branch contains checked and tried works that will be used in production, while the development branch is for merging and testing the changes. The work is continued in short-lived branches, which are transformed back to the development branch. Davies (2024) mentions that the features of commit history, version comparison, and reverting contribute to the flexibility and accountability of the team, while open repositories provide different participants with the opportunity to work at the same time without any damage to the project. The history traced in this case may also help to find out about security-sensitive designs, that is quite useful for the design process. Summers (2020) also notes that the integrated team contributes to the success of the team, as there are coordinated actions on behalf of different specialists.

The planned instruments for implementation were: Python and pandas for data generation and processing, SQLite for storing data locally, and Streamlit or Dash for creating the dashboard; however, already at Section 4.4 the implementation will be presented in Python. ProjectLibre was chosen for breakdown and path calculations among various open-source tools mentioned in the survey by Bergmann (2021).

RESULTS AND EVALUATION

This chapter reports what was built and verified across Units 4 to 7. The project's goal, set in Chapter 1, is to catch a silent home monitoring device early without flooding nurses with false alarms. The system therefore judges each device against its own usual rhythm instead of one fixed time limit, places patients who need a call in a ranked queue, and records what each call found. The sections below describe the core algorithm, the integration of all modules into one application, and the tests that verify both. All data in this chapter is synthetic.

4.1 Milestones Achieved

As of the close of Unit 3, three milestones have been achieved. The first of these was project definition, comprising the identification of the aim, the boundaries and exclusions of the project, preliminary literature review, and tools for project implementation including a Gantt chart identifying the critical path, resource justification table, and feasibility checklist covering the technical, operational, and financial aspects.

The second milestone was the formulation of the problem statement and project plan that included stakeholder analysis, deadlines for measurable objectives, ethical and security analysis, and a strategy of risk mitigation, which used synthetic data as risk-avoidance means instead of a convenient solution.

The last milestone to be reached in this unit was detailed design accomplished with the means of four-module layered architecture specifying inputs, outputs, and method applied for each module, eight functional requirements, and seven non-functional requirements linked to each design decision, in addition to the construction of the system architecture diagram.

4.2 Development Across Units 4 to 7

As detailed in Table 1, four distinct versions of the system were developed. Versions 0.1.0 and 0.2.0 are designated releases found in the GitHub repository at https://github.com/AnamKazani/rpm-gap-detection, and they provide a summary of what was modified. Version 0.3.0 implements the contribution of Unit 6 into the system. In Unit 4, a Python file was developed which executed all four modules using synthetic data, and the results were shown in the video. In Unit 5, the scoring rule was separated from the database and timer to enable the testing of its functionality and the identification of four faults, which were successfully fixed. Lastly, Unit 6 retained the integration in a single file and developed the application with the API and nursing dashboard feature, followed by the assessment of its precision and speed of execution. Testing of the whole system is incorporated in Unit 7, while version 0.4.0 demonstrates the eliminated imperfections found in testing.

Table 1

Versions and Milestones

Version	Unit	Milestone	Main evidence
v0.1.0	4	First working system: four modules in one file	Demonstration video
v0.2.0	5	Testable core logic and 21 unit tests	Test runs and four fixed defects
v0.3.0	6	Web API, dashboard, and evaluation in the same file	Seven integration checks and evaluation results

Version	Unit	Milestone	Main evidence
v0.4.0	7	System, acceptance, regression, and performance testing	46 test cases and four more fixed defects

4.3 Core Algorithm Implementation

A central function of the algorithm is to know if the device is silent for longer than it usually is. Just having a rule that tells the algorithm to flag devices that have been silent for a period of 48 hours is inefficient because it will flag people who send readings only every few days and will be slow to identify devices that send readings twice a day. The algorithm therefore learns the individual time pattern of each device.

In Module 2, two small functions operate (see Figure 2). The first one is called usual_gap(), and it takes the readings that have been taken with an interval of up to one hour as belonging to one session because some blood pressure cuffs are able to take several readings in a few minutes. In addition, it takes the median time gap over the last 30 days, rather than the average, as an outlier such as a holiday would affect the average value. At least five time intervals have to be registered for the device to start relying on its specific time pattern. A default gap of 24 hours is used otherwise. The second function is called score_device() and computes the ratio of the time that the device has been silent now to its usual gap. A device gets flagged whenever the value of this ratio reaches 2.0 and the period of silence lasts at least 24 hours; hence, a device that usually produces readings every six hours cannot get flagged after remaining silent for only twelve hours. Each result comes with a reason code.

Two patients from the demonstration data show how the rule behaves. Patient P-1014's device usually sends every 12.1 hours and had been silent for 33.1 hours, a score of 2.7, so it was

flagged even though a 48-hour rule would still have treated it as normal. Patient P-1029's device usually sends every 74.2 hours and had been silent for 242 hours, a score of 3.3. A device like P-1029's that had been quiet for 60 hours would score about 0.8 and stay off the queue, while a 48-hour rule would flag it.

Module 1, ingestion, protects these scores. It keeps only three fields from each vendor record (patient, device, and reading time), skips patients without consent, and rejects reading times that are missing, unreadable, or later than the time of scoring. A repeated reading is stored once, because the database key includes all three fields. Module 3 builds the outreach queue from flagged devices in the nurse's own program, highest score first, and leaves out patients whose calls have been logged. Module 4 records each call outcome from a fixed list, refuses calls for patients outside the nurse's program, and uses a database trigger so that a logged outcome cannot be edited (Figure 3).

The Unit 5 unit tests shaped this design. Following Rana (2023), each of the 21 test cases has an ID and an expected result, and the suite combines black-box tests written from the requirements with white-box tests written from the code's branches, two of the types described by GeeksforGeeks (2025c). Amazon Web Services (n.d.) recommends that unit tests check logic, boundaries, and error handling, and the first run failed six tests that pointed to four defects. An unreadable reading time stopped scoring for every patient, and a future-dated reading gave a device that had been silent for nine days a negative silence that removed it from the queue. Scoring grouped readings by patient, so a working scale could hide a silent cuff. Bursts of cuff readings pulled the usual gap down to minutes, which made an ordinary day of silence look like an outage. Finally, an unknown user crashed the queue, and a nurse could log a call for another

program's patient. Each defect was fixed and covered by a test, and the results on the synthetic data did not change.

Figure 2

Main Logic of Modules 1 and 2

```
def clean_time(text, as_of):
    """Return the reading time in one standard format, or None if it is
    missing, unreadable, or later than as_of."""
    try:
        when = datetime.fromisoformat(text)
        if when > as_of:
            return None
        return when.isoformat(timespec="seconds")
    except (TypeError, ValueError):
        return None


def ingest(db, vendor_rows, consented, as_of):
    """Keep only patient, device and time, for consented patients."""
    counts = {"total": 0, "no_consent": 0, "bad_time": 0, "stored": 0}
    for row in vendor_rows:
        counts["total"] += 1
        when = clean_time(row["reading_time"], as_of)
        if row["patient_id"] not in consented:
            counts["no_consent"] += 1
        elif when is None:
            counts["bad_time"] += 1
        else:
            counts["stored"] += db.execute(
                "INSERT OR IGNORE INTO readings VALUES (?, ?, ?)",
                (row["patient_id"], row["device_id"], when)).rowcount
    counts["duplicates"] = (counts["total"] - counts["no_consent"]
                            - counts["bad_time"] - counts["stored"])
    return counts


# ---------------------------------------------------- Module 2: gap scoring
def usual_gap(times):
    """Median hours between sessions over the last 30 days. Readings less
    than an hour apart count as one session. None if there is no pattern yet."""
    times = sorted(times)
    sessions = [t for i, t in enumerate(times)
                if i == 0 or t - times[i - 1] > timedelta(minutes=SESSION_MINUTES)]
    recent = [t for t in sessions if t >= sessions[-1] - timedelta(days=BASELINE_DAYS)]
    gaps = [(b - a).total_seconds() / 3600 for a, b in zip(recent, recent[1:])]
    if len(gaps) < MIN_GAPS:
        return None
    return statistics.median(gaps)


def score_device(times, as_of, enrolled_on):
    """How many usual gaps the device has been silent for, as of a given time."""
    times = [t for t in times if t <= as_of]          # ignore future readings
    last = max(times) if times else enrolled_on       # never sent? count from enrollment
    silent = (as_of - last).total_seconds() / 3600
    usual = usual_gap(times)
    score = silent / (usual or DEFAULT_GAP_HOURS)
    flagged = score >= FLAG_AT and silent >= MIN_SILENT_HOURS
    if usual is None:
        reason = "INSUFFICIENT_HISTORY"
    elif flagged:
        reason = "SILENCE_EXCEEDS_BASELINE"
    else:
        reason = "WITHIN_PERSONAL_PATTERN"
    return {"silent": silent, "usual": usual, "score": score,
            "reason": reason, "flagged": flagged}
```

Figure 3

Main Logic of Modules 3 and 4

```
178
179 # ---------------------------------------------------- Module 3: outreach queue
180 def outreach_queue(db, nurse):
181     """Flagged patients in the nurse's own program, highest score first."""
182     program = NURSES.get(nurse)
183     if program is None:                    # unknown users get nothing
184         return []
185     return db.execute("""
186         SELECT s.patient_id, s.device_id, s.silent, s.usual, s.score, s.reason
187         FROM scores s JOIN patients p USING (patient_id, device_id)
188         WHERE s.flagged = 1 AND p.program = ?
189           AND s.patient_id NOT IN (SELECT patient_id FROM calls)
190         ORDER BY s.score DESC, s.silent DESC""", (program,)).fetchall()
191
192
193 def explain(silent, usual, score, reason):
194     """Turn a reason code into a sentence a nurse can check before calling."""
195     if reason == "INSUFFICIENT_HISTORY":
196         return (f"New device: silent {silent:.0f} h. Judged against the default "
197                 f"{DEFAULT_GAP_HOURS} h gap until it has a pattern.")
198     return f"Silent {silent:.0f} h, which is {score:.1f} times its usual {usual:.0f} h gap."
199
200
201 # ---------------------------------------------------- Module 4: call log
202 def log_outcome(db, nurse, patient_id, outcome):
203     """Save what the call found. Nurses can only log their own patients."""
204     if outcome not in OUTCOMES:
205         raise ValueError(f"unknown outcome: {outcome}")
206     row = db.execute("SELECT program FROM patients WHERE patient_id = ?",
207                      (patient_id,)).fetchone()
208     if row is None or row[0] != NURSES.get(nurse):
209         raise PermissionError(f"{nurse} cannot log calls for {patient_id}")
210     db.execute("INSERT INTO calls VALUES (?, ?, ?, ?)",
211                (patient_id, outcome, nurse, datetime.now().isoformat(" ", "minutes")))
```

4.4 System Integration Summary

Version 0.3.0 adds a web layer to the same file, gap_detection.py, that turns the four modules into one running application. Data moves in one direction through the backend: the vendor file is saved and read back as a CSV file, Module 1 stores the cleaned readings in a SQLite database, and Module 2 scores every device and saves the scores. The browser dashboard then reads the queue from Module 3 and sends call outcomes to Module 4 through a small web API, listed in Table 2. Like the earlier versions, the application uses only Python's standard library. The Unit 3 design had planned a dashboard framework, but the standard library's web server was enough for this scope and keeps the system free of installed dependencies, which also makes deployment simpler.

Table 2

Web API of the Integrated System

Endpoint	Module	Purpose	Main responses
POST /api/login	3	Sign in a nurse and return a token	200; 401 unknown user
GET /api/queue	3	Return the nurse's ranked queue	200; 401 missing or unknown token
POST /api/calls	4	Log what a call found	201; 400 bad outcome; 403 other program
GET /api/health	All	Report that the service is running	200

Just one command, python3 src/gap_detection.py, runs a complete five-step process, and the second step of this process is illustrated in Figure 4. The first step of the program is creating a synthetic vendor file consisting of 1,148 readings that include blood pressure and pulse fields, with one time in the wrong format and one time two days in the future. Ingestion accepted 1,045 readings in total, rejecting or skipping the rest of the rows, which came from patients without consent, had a bad time, or duplicated a reading. There are 28 devices in total that were checked through scoring, of which eight were flagged. The answer key confirms that all seven quiet devices and none of the 20 normal devices were flagged. All the parameters important for the program such as the address, port, and flag threshold are defined in the environment variables described in Section 5.2. Ingestion only records readings that have never been recorded before, so if the same file is processed again, nothing is added. In real use, the vendor’s daily export would replace the synthetic file, and the whole process would run the same way.

Several decisions were made in the design of the software to ensure its safety. When users log in, the server creates a random token for every user and remembers which nurse each token belongs to. Scoring and ingestion happen in one database transaction, so the queue never shows incomplete scores, and a lock lets only one request use the database at a time.

Figure 5 shows the API answering real requests. Nurse Alvarez signs in and receives five patients ranked by score. The first, P-1007, has been silent for 225 hours against a usual gap of 23 hours, a score of 9.7. Logging a call for that patient returns status 201 and leaves four patients in her queue. A call for a Program B patient returns status 403, a request without a token returns 401, and a direct attempt to edit the logged call in the database is blocked. Figure 6 shows the dashboard afterward, with the four remaining patients, where each reason code appears with a plain sentence, such as "Silent 89 h, which is 3.8 times its usual 24 h gap."

The dashboard was kept simple on purpose. Each row shows the rank, patient, device, hours of silence, usual gap, and score. The score badge is red at 4.0 or more, amber from 2.0, and gray for a new device, and the number is always printed so that color is never the only signal. Outcomes are chosen from a fixed list, and a footer reminds nurses that a logged call is final.

Two code excerpts show how the parts communicate. On the browser side, every dashboard action passes through one JavaScript function, api(), which attaches the token, sends the request, and turns any error reply into a message on the screen (Figure 7). On the server side, the queue handler checks the token, reads the saved scores through Module 3, and adds a plain-language reason to each row, while run_pipeline() runs Modules 1 and 2 in one transaction (Figure 8).

Figure 4

Ingestion and Scoring Output

```
STEP 2  Modules 1 and 2: ingestion and gap scoring (saved in SQLite)
----------------------------------------------------------------------
  Rows read               1,148
  Stored                  1,045  (patient, device and time only)
  Skipped, no consent        90
  Rejected, bad time          2
  Duplicates ignored         11
  Devices scored 28, flagged 8 (score of 2.0 or more)
  Answer key: 7 of 7 quiet devices flagged, 0 of 20 normal devices flagged

(press Enter for the next step)
```

Figure 5

API Requests and Replies

```
STEP 3  Modules 3 and 4 through the web API (real HTTP requests)
----------------------------------------------------------------
  > POST /api/login  {"nurse": "Nurse Alvarez"}
  < 200  signed in, token 36435e28...
  > GET  /api/queue  (with token)
  < 200  5 patients for Program A, highest score first:
      1  P-1007  score  9.7  Silent 225 h, which is 9.7 times its usual 23 h gap.
      2  P-1008  score  3.8  Silent 89 h, which is 3.8 times its usual 24 h gap.
      3  P-1029  score  3.3  Silent 242 h, which is 3.3 times its usual 74 h gap.
      4  P-1014  score  2.7  Silent 33 h, which is 2.7 times its usual 12 h gap.
      5  P-1006  score  2.4  New device: silent 57 h. Judged against the default 24 h gap until it has a pattern.
  > POST /api/calls  {"patient_id": "P-1007", "outcome": "device failure"}
  < 201  {"logged": "P-1007", "left_in_queue": 4}
  > POST /api/calls  {"patient_id": "P-1021", "outcome": "resolved"}
  < 403  {"error": "Nurse Alvarez cannot log calls for P-1021"}
  > GET  /api/queue  (no token)
  < 401  {"error": "sign in first"}
  Editing the logged call in the database: BLOCKED (call outcomes cannot be changed)
  > GET  /api/health
  < 200  {"status": "ok", "version": "0.3.0"}
  Integration checks passed: 7 of 7

(press Enter for the next step)
```

Figure 6

Nurse Dashboard After Sign-In

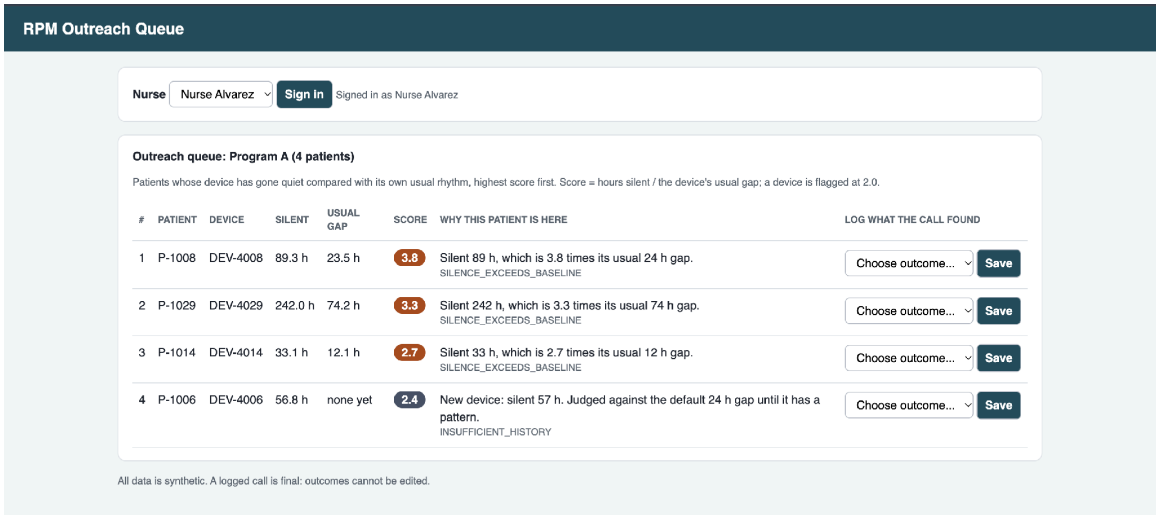


Figure 7

Dashboard-to-API Link

```
// Every screen action goes through this one function: browser -> API -> database.
async function api(method, path, body) {
  const response = await fetch(path, { method, body: body && JSON.stringify(body),
    headers: { "Content-Type": "application/json", "Authorization": "Bearer " + token } });
  const data = await response.json();
  if (!response.ok) throw new Error(data.error);
  return data;
}

function show(text, kind) { $("message").textContent = text; $("message").className = kind; }

async function signIn() {
  const user = await api("POST", "/api/login", { nurse: $("nurse").value });
  token = user.token;
  $("who").textContent = "Signed in as " + user.nurse;
  show("", "");
  loadQueue();
}

async function loadQueue() {
  const data = await api("GET", "/api/queue");
  $("title").textContent = `Outreach queue: ${data.program} (${data.patients.length} patients)`;
  const options = data.outcomes.map((o) => `<option>${o}</option>`).join("");
  $("rows").innerHTML = data.patients.map((p) => `<tr>
    <td>${p.rank}</td><td>${safe(p.patient_id)}</td><td>${safe(p.device_id)}</td>
    <td>${p.silent_hours.toFixed(1)} h</td><td>${p.usual_hours ? p.usual_hours.toFixed(1) + " h" : "none yet"}</td>
    <td><span class="score" style="background:${p.reason === "INSUFFICIENT_HISTORY" ? "#475467"
      : p.score >= 4 ? "#b42318" : "#b54708"}">${p.score.toFixed(1)}</span></td>
    <td class="why">${safe(p.explanation)}<small>${p.reason}</small></td>
    <td><select id="pick-${p.rank}"><option value="">Choose outcome...</option>${options}</select>
      <button onclick="logCall('${safe(p.patient_id)}', $('pick-${p.rank}').value)">Save</button></td>
  </tr>`).join("") || `<tr><td colspan="8" class="hint">No patients need a call right now.</td></tr>`;
}

async function logCall(patientId, outcome) {
  if (!outcome) return show(`Choose what the call found for ${patientId} before saving.`, "error");
  try {
    const result = await api("POST", "/api/calls", { patient_id: patientId, outcome });
    show(`Call logged for ${patientId} (${outcome}). ${result.left_in_queue} patients left in your queue.`, "ok");
    loadQueue();
  } catch (error) { show("Not saved: " + error.message, "error"); }
}
</script></body></html>
```

Figure 8

API, Module, and Database Links

```


  def run_pipeline(db, enrollment, as_of):
      """Modules 1 and 2 together: read the vendor file, store the readings and
      score every device, all in one database transaction."""
      with open(VENDOR_FILE, newline="") as f:
          vendor_rows = list(csv.DictReader(f))
      with db:
          counts = ingest(db, vendor_rows, load_roster(db, enrollment), as_of)
          score_devices(db, as_of)
      return counts


  def check_answer_key(db, answer_key):
      """Output validation: compare the flags with what each synthetic patient
      was built to be. Returns {situation: [devices, flagged, 48-hour rule]}."""
```

```
221
222     def do_GET(self):
223         if self.path == "/":
224             return self.reply(200, DASHBOARD.encode(), "text/html; charset=utf-8")
225         if self.path == "/api/health":
226             return self.send_json(200, {"status": "ok", "version": VERSION})
227         if self.path == "/api/queue":
228             nurse = self.signed_in_nurse()
229             if nurse is None:
230                 return self.send_json(401, {"error": "sign in first"})
231             with LOCK:
232                 queue = outreach_queue(self.db, nurse)                    # Module 3
233             patients = [{"rank": rank, "patient_id": p, "device_id": d,
234                          "silent_hours": round(s, 1), "usual_hours": round(u, 1) if u else None,
235                          "score": round(sc, 1), "reason": r, "explanation": explain(s, u, sc, r)}
236                         for rank, (p, d, s, u, sc, r) in enumerate(queue, 1)]
237             return self.send_json(200, {"program": NURSES[nurse], "patients": patients,
238                                         "outcomes": OUTCOMES})
239         self.send_json(404, {"error": "not found"})
240
```

4.5 Verification of the Integrated System

Unit tests check each module alone, but they cannot show that modules work together (GeeksforGeeks, 2025c). Summers (2020) describes assembly integration as combining modules and testing their compatibility, so the third step of the walkthrough runs seven integration checks against the running application over real HTTP. They confirm that sign-in works, that the queue arrives with five patients in score order, that a logged call is saved and leaves the queue, and that a call for another program returns 403. They also confirm that a request without a token returns 401, that the database blocks any edit to a logged call, and that the health check answers. All seven checks pass, and the 21 unit tests from Unit 5 still pass against the new file (Figure 9).

Several of these checks are the executable form of Unit 3 requirements. The checks for missing tokens and cross-program calls, together with the unit tests for unknown users, test NFR-2, which requires server-side authorization and a test showing that a nurse in one program cannot reach another program's patients. The duplicate-reading unit test checks NFR-5, which requires ingestion to be safe to repeat. Krüger (2025) explains that a requirements specification makes expectations explicit and verifiable before code exists, and these tests show that payoff.

Summers (2020) separates verification, which confirms that a product matches its specification, from validation, which confirms that it meets users' needs. The tests and checks above are verification. The evaluation in Section 4.7 is the first step of validation.

Figure 9

Unit Tests Passing Against Version 0.3.0



4.6 System Testing

System Testing

The 21 unit tests from Unit 5 still check each module under test. A further testing source file, test_system.py, was built for system-level testing, where another 25 test cases at four additional levels, plus regression tests, were created by me. In the integration tests, I tested the modules and database based on their ability to exchange data. The system tests checked the application as a black box by sending it real requests through HTTP to evaluate its functional and non-functional capabilities, which is how Hrupa (2024) and GeeksforGeeks (2025b) describe system testing. Acceptance tests directly assessed the Unit 3 functional requirements from the perspective of the nurse. Regression tests confirmed that the Unit 6 accuracy and detection times did not change,

and performance tests checked speed and stability in an environment that resembles production, as BrowserStack (2025) advises.

The first run against version 0.3.0 passed 41 tests and failed four (Figures 10 and 11). One test, for the FR-8 cohort report, is marked as an expected failure because that report has not been built yet.

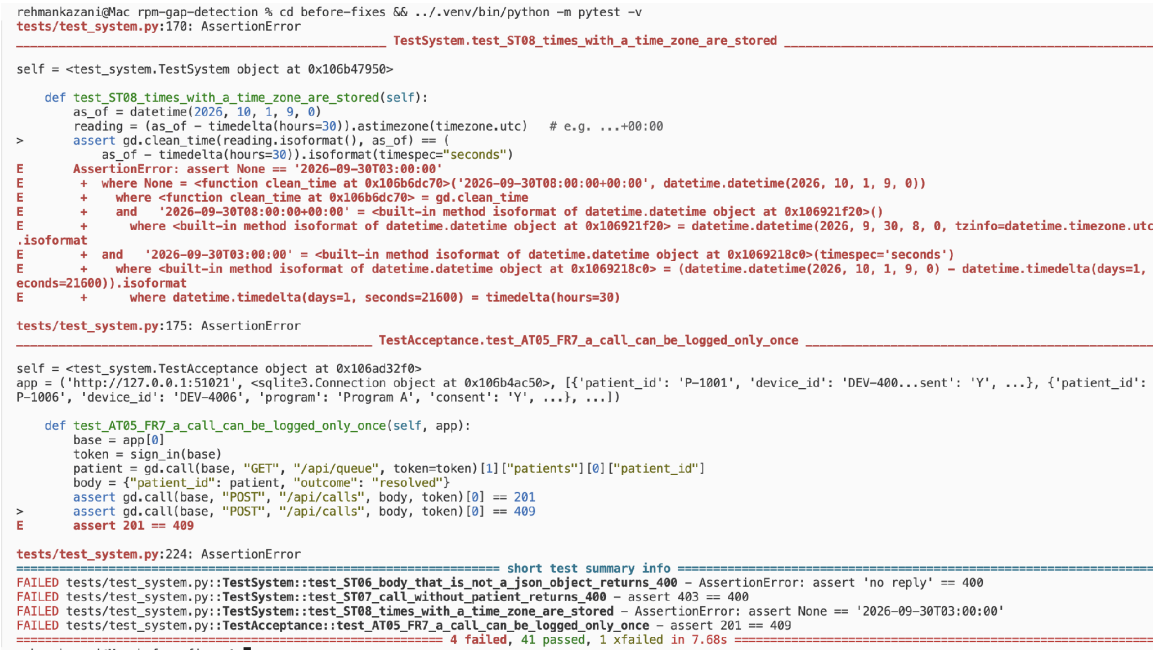
Figure 10

First Test Run Against Version 0.3.0

```
tests/test_gap_detection.py::TestBlackBox::test_S01_daily_device_quiet_10h_is_normal PASSED                                  [  2%]
tests/test_gap_detection.py::TestBlackBox::test_S02_quiet_two_usual_gaps_is_flagged PASSED                                   [  4%]
tests/test_gap_detection.py::TestBlackBox::test_S03_quiet_47h_is_not_flagged PASSED                                          [  6%]
tests/test_gap_detection.py::TestBlackBox::test_S04_rural_device_quiet_3_5_days_is_normal PASSED                             [  8%]
tests/test_gap_detection.py::TestBlackBox::test_S05_twice_daily_device_quiet_30h_is_flagged PASSED                           [ 10%]
tests/test_gap_detection.py::TestBlackBox::test_S06_needs_a_full_day_of_silence PASSED                                       [ 13%]
tests/test_gap_detection.py::TestBlackBox::test_S07_burst_of_readings_is_one_session PASSED                                  [ 15%]
tests/test_gap_detection.py::TestBlackBox::test_I01_only_consented_patients_and_3_fields PASSED                              [ 17%]
tests/test_gap_detection.py::TestBlackBox::test_I02_unreadable_or_future_time_not_stored PASSED                              [ 19%]
tests/test_gap_detection.py::TestBlackBox::test_Q01_own_program_only_highest_score_first PASSED                              [ 21%]
tests/test_gap_detection.py::TestBlackBox::test_Q02_unknown_user_gets_empty_queue PASSED                                     [ 23%]
tests/test_gap_detection.py::TestBlackBox::test_L01_cannot_log_call_for_other_program PASSED                                 [ 26%]
tests/test_gap_detection.py::TestBlackBox::test_L02_unknown_outcome_refused PASSED                                           [ 28%]
tests/test_gap_detection.py::TestBlackBox::test_E01_flags_match_synthetic_answer_key PASSED                                  [ 30%]
tests/test_gap_detection.py::TestWhiteBox::test_W01_no_readings_counts_from_enrollment PASSED                                [ 32%]
tests/test_gap_detection.py::TestWhiteBox::test_W02_pattern_needs_five_gaps PASSED                                           [ 34%]
tests/test_gap_detection.py::TestWhiteBox::test_W03_reading_after_as_of_is_ignored PASSED                                    [ 36%]
tests/test_gap_detection.py::TestWhiteBox::test_W04_quiet_device_beside_working_one PASSED                                   [ 39%]
tests/test_gap_detection.py::TestWhiteBox::test_W05_resent_reading_stored_once PASSED                                        [ 41%]
tests/test_gap_detection.py::TestWhiteBox::test_W06_logged_call_leaves_queue_and_cannot_change PASSED                        [ 43%]
tests/test_gap_detection.py::TestWhiteBox::test_W07_demo_runs_to_the_end PASSED                                              [ 45%]
tests/test_system.py::TestIntegration::test_IT01_login_then_queue_in_score_order PASSED                                      [ 47%]
tests/test_system.py::TestIntegration::test_IT02_queue_shows_what_scoring_saved PASSED                                       [ 50%]
tests/test_system.py::TestIntegration::test_IT03_logged_call_is_saved_and_leaves_queue PASSED                                [ 52%]
tests/test_system.py::TestIntegration::test_IT04_failed_scoring_leaves_no_half_saved_data PASSED                             [ 54%]
tests/test_system.py::TestIntegration::test_IT05_reading_the_same_file_twice_adds_nothing PASSED                             [ 56%]
tests/test_system.py::TestSystem::test_ST01_full_walkthrough_passes_all_checks PASSED                                        [ 58%]
tests/test_system.py::TestSystem::test_ST02_health_check_reports_version PASSED                                              [ 60%]
tests/test_system.py::TestSystem::test_ST03_unknown_address_returns_404 PASSED                                               [ 63%]
tests/test_system.py::TestSystem::test_ST04_missing_or_fake_token_returns_401 PASSED                                         [ 65%]
tests/test_system.py::TestSystem::test_ST05_other_programs_patient_returns_403 PASSED                                        [ 67%]
tests/test_system.py::TestSystem::test_ST06_body_that_is_not_a_json_object_returns_400 FAILED                                [ 69%]
tests/test_system.py::TestSystem::test_ST07_call_without_patient_returns_400 FAILED                                          [ 71%]
tests/test_system.py::TestSystem::test_ST08_times_with_a_time_zone_are_stored FAILED                                         [ 73%]
tests/test_system.py::TestSystem::test_ST09_patient_data_is_never_cached PASSED                                              [ 76%]
tests/test_system.py::TestAcceptance::test_AT01_FR2_only_three_fields_are_kept PASSED                                        [ 78%]
tests/test_system.py::TestAcceptance::test_AT02_FR3_no_consent_never_reaches_a_nurse PASSED                                  [ 80%]
tests/test_system.py::TestAcceptance::test_AT03_FR5_every_row_explains_itself PASSED                                         [ 82%]
tests/test_system.py::TestAcceptance::test_AT04_FR6_each_nurse_sees_only_her_program PASSED                                  [ 84%]
tests/test_system.py::TestAcceptance::test_AT05_FR7_a_call_can_be_logged_only_once FAILED                                    [ 86%]
tests/test_system.py::TestAcceptance::test_AT06_FR8_cohort_report_exists XFAIL (FR-8 cohort report is not built yet (planned for a later version))  [ 89%]
tests/test_system.py::TestRegression::test_RT01_accuracy_on_240_patients_is_unchanged PASSED                                 [ 91%]
tests/test_system.py::TestRegression::test_RT02_detection_times_are_unchanged PASSED                                         [ 93%]
tests/test_system.py::TestPerformance::test_PT01_NFR3_4800_patients_scored_well_within_an_hour PASSED                        [ 95%]
tests/test_system.py::TestPerformance::test_PT02_queue_answers_95_percent_within_200_ms PASSED                               [ 97%]
tests/test_system.py::TestPerformance::test_PT03_50_requests_at_once_all_succeed PASSED                                     [100%]
```

Figure 11

Why the Four Tests Failed



I fixed each defect in version 0.4.0 (Figure 12) and ran every test again. All 45 tests now pass, with the one expected failure (Figure 13). The performance tests ingested and scored 4,800 patients in 0.79 seconds, and the queue API answered 95% of 100 requests within 1.0 millisecond (Figure 14).

Figure 12

Fix for Defects D2 and D4 in log_outcome()

```
204   class AlreadyLogged(Exception):
205       """A call outcome is already saved for this patient."""
206
207
208   def log_outcome(db, nurse, patient_id, outcome):
209       """Save what the call found. Nurses can only log their own patients,
210       and only once per patient, because a logged call is final."""
211       if not patient_id:
212           raise ValueError("patient_id is required")
213       if outcome not in OUTCOMES:
214           raise ValueError(f"unknown outcome: {outcome}")
215       row = db.execute("SELECT program FROM patients WHERE patient_id = ?",
216                        (patient_id,)).fetchone()
217       if row is None or row[0] != NURSES.get(nurse):
218           raise PermissionError(f"{nurse} cannot log calls for {patient_id}")
219       if db.execute("SELECT 1 FROM calls WHERE patient_id = ?", (patient_id,)).fetchone():
220           raise AlreadyLogged(f"a call is already logged for {patient_id}")
221       db.execute("INSERT INTO calls VALUES (?, ?, ?, ?)",
222                  (patient_id, outcome, nurse, datetime.now().isoformat(" ", "minutes")))
223
```

Figure 13

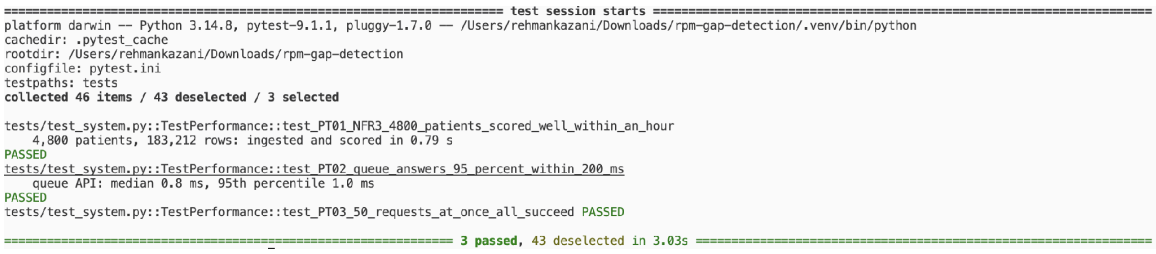
Second Test Run After the Fixes (Version 0.4.0)

```
tests/test_gap_detection.py::TestBlackBox::test_S01_daily_device_quiet_10h_is_normal PASSED                                  [  2%]
tests/test_gap_detection.py::TestBlackBox::test_S02_quiet_two_usual_gaps_is_flagged PASSED                                   [  4%]
tests/test_gap_detection.py::TestBlackBox::test_S03_quiet_47h_is_not_flagged PASSED                                          [  6%]
tests/test_gap_detection.py::TestBlackBox::test_S04_rural_device_quiet_3_5_days_is_normal PASSED                             [  8%]
tests/test_gap_detection.py::TestBlackBox::test_S05_twice_daily_device_quiet_30h_is_flagged PASSED                           [ 10%]
tests/test_gap_detection.py::TestBlackBox::test_S06_needs_a_full_day_of_silence PASSED                                       [ 13%]
tests/test_gap_detection.py::TestBlackBox::test_S07_burst_of_readings_is_one_session PASSED                                  [ 15%]
tests/test_gap_detection.py::TestBlackBox::test_I01_only_consented_patients_and_3_fields PASSED                              [ 17%]
tests/test_gap_detection.py::TestBlackBox::test_I02_unreadable_or_future_time_not_stored PASSED                              [ 19%]
tests/test_gap_detection.py::TestBlackBox::test_Q01_own_program_only_highest_score_first PASSED                              [ 21%]
tests/test_gap_detection.py::TestBlackBox::test_Q02_unknown_user_gets_empty_queue PASSED                                     [ 23%]
tests/test_gap_detection.py::TestBlackBox::test_L01_cannot_log_call_for_other_program PASSED                                 [ 26%]
tests/test_gap_detection.py::TestBlackBox::test_L02_unknown_outcome_refused PASSED                                           [ 28%]
tests/test_gap_detection.py::TestBlackBox::test_E01_flags_match_synthetic_answer_key PASSED                                  [ 30%]
tests/test_gap_detection.py::TestWhiteBox::test_W01_no_readings_counts_from_enrollment PASSED                                [ 32%]
tests/test_gap_detection.py::TestWhiteBox::test_W02_pattern_needs_five_gaps PASSED                                           [ 34%]
tests/test_gap_detection.py::TestWhiteBox::test_W03_reading_after_as_of_is_ignored PASSED                                    [ 36%]
tests/test_gap_detection.py::TestWhiteBox::test_W04_quiet_device_beside_working_one PASSED                                   [ 39%]
tests/test_gap_detection.py::TestWhiteBox::test_W05_resent_reading_stored_once PASSED                                        [ 41%]
tests/test_gap_detection.py::TestWhiteBox::test_W06_logged_call_leaves_queue_and_cannot_change PASSED                        [ 43%]
tests/test_gap_detection.py::TestWhiteBox::test_W07_demo_runs_to_the_end PASSED                                              [ 45%]
tests/test_system.py::TestIntegration::test_IT01_login_then_queue_in_score_order PASSED                                      [ 47%]
tests/test_system.py::TestIntegration::test_IT02_queue_shows_what_scoring_saved PASSED                                       [ 50%]
tests/test_system.py::TestIntegration::test_IT03_logged_call_is_saved_and_leaves_queue PASSED                                [ 52%]
tests/test_system.py::TestIntegration::test_IT04_failed_scoring_leaves_no_half_saved_data PASSED                             [ 54%]
tests/test_system.py::TestIntegration::test_IT05_reading_the_same_file_twice_adds_nothing PASSED                             [ 56%]
tests/test_system.py::TestSystem::test_ST01_full_walkthrough_passes_all_checks PASSED                                        [ 58%]
tests/test_system.py::TestSystem::test_ST02_health_check_reports_version PASSED                                              [ 60%]
tests/test_system.py::TestSystem::test_ST03_unknown_address_returns_404 PASSED                                               [ 63%]
tests/test_system.py::TestSystem::test_ST04_missing_or_fake_token_returns_401 PASSED                                         [ 65%]
tests/test_system.py::TestSystem::test_ST05_other_programs_patient_returns_403 PASSED                                        [ 67%]
tests/test_system.py::TestSystem::test_ST06_body_that_is_not_a_json_object_returns_400 PASSED                                [ 69%]
tests/test_system.py::TestSystem::test_ST07_call_without_patient_returns_400 PASSED                                          [ 71%]
tests/test_system.py::TestSystem::test_ST08_times_with_a_time_zone_are_stored PASSED                                         [ 73%]
tests/test_system.py::TestSystem::test_ST09_patient_data_is_never_cached PASSED                                              [ 76%]
tests/test_system.py::TestAcceptance::test_AT01_FR2_only_three_fields_are_kept PASSED                                        [ 78%]
tests/test_system.py::TestAcceptance::test_AT02_FR3_no_consent_never_reaches_a_nurse PASSED                                  [ 80%]
tests/test_system.py::TestAcceptance::test_AT03_FR5_every_row_explains_itself PASSED                                         [ 82%]
tests/test_system.py::TestAcceptance::test_AT04_FR6_each_nurse_sees_only_her_program PASSED                                  [ 84%]
tests/test_system.py::TestAcceptance::test_AT05_FR7_a_call_can_be_logged_only_once PASSED                                    [ 86%]
tests/test_system.py::TestAcceptance::test_AT06_FR8_cohort_report_exists XFAIL (FR-8 cohort report is not built yet (planned for a later version))  [ 89%]
tests/test_system.py::TestRegression::test_RT01_accuracy_on_240_patients_is_unchanged PASSED                                 [ 91%]
tests/test_system.py::TestRegression::test_RT02_detection_times_are_unchanged PASSED                                         [ 93%]
tests/test_system.py::TestPerformance::test_PT01_NFR3_4800_patients_scored_well_within_an_hour PASSED                        [ 95%]
tests/test_system.py::TestPerformance::test_PT02_queue_answers_95_percent_within_200_ms PASSED                               [ 97%]
tests/test_system.py::TestPerformance::test_PT03_50_requests_at_once_all_succeed PASSED                                     [100%]

========================================================== 45 passed, 1 xfailed in 7.71s ===========================================================
```

Figure 14

Performance Test Output



Test Documentation

Testing aimed at determining whether version 0.3.0 satisfies the Unit 3 specifications as a complete system before getting ready for deployment. It included testing of all four modules, web API, and database with synthetic data on Python 3.14 on a Mac. Following GeeksforGeeks (2025b), the process included the following steps: writing test cases from requirements, preparing test data, executing tests, recording defects, correcting them, and executing regression tests once again. Every test case is listed by its ID in Figures 10 and 13, and the results and defects are summarized in Tables 3 and 4, respectively.

Table 3

Test Summary Report

Level	Tests	What it checks	First run (v0.3.0)	After fixes (v0.4.0)
Unit	21	Each module alone	21 passed	21 passed
Integration	5	Modules and database together	5 passed	5 passed

Level	Tests	What it checks	First run (v0.3.0)	After fixes (v0.4.0)
System	9	Whole running application over HTTP	6 passed, 3 failed	9 passed
Acceptance	6	Functional requirements FR-2 to FR-8	4 passed, 1 failed, 1 expected failure	5 passed, 1 expected failure
Regression	2	Unit 6 results unchanged	2 passed	2 passed
Performance	3	Speed and stability	3 passed	3 passed
Total	46		41 passed, 4 failed, 1 expected failure	45 passed, 1 expected failure

Table 4

Defects Found and Fixed

ID	Test	Problem in version 0.3.0	Fix in version 0.4.0
D1	ST06	A request body that was not a JSON object, such as [], crashed the request, and the server closed the connection without replying	The API checks the body and answers 400
D2	ST07	Logging a call with no patient ID returned 403 instead of 400	A patient ID is required

ID	Test	Problem in version 0.3.0	Fix in version 0.4.0
D3	ST08	Reading times with a time zone, such as +00:00, were rejected, so a vendor sending UTC times would lose every reading	Times with a zone are converted to local time
D4	AT05	The same call could be logged twice, giving one patient two outcomes	A second call returns 409

While fixing these, I also made the dashboard escape apostrophes in patient IDs. Some limitations remain documented rather than fixed. The FR-8 cohort report is missing, usability has not been tested with nurses, the database is kept in memory, and devices that normally send every three days still take a median of 142 hours to flag.

Evaluation

The unit tests were successful; however, system testing discovered four defects. The defects occurred on the boundaries where the system interacts with the vendor files and client requests, a situation where there is no single module accountable. Hrupa (2024) claims that system testing is the final structured check before users judge a product; D3 shows exactly that, as on the first day of using a new vendor it would have simply rejected every real reading. The regression tests demonstrated that the modifications made no difference in the outputs, as the result is still 1.00 for precision and recall, and the detection times remain unchanged at 25, 49, and 142 hours. Writing each test based on a requirement means that all the outputs can be traced back to the initial design, which results in easier system acceptance and support.

4.7 Evaluation Metrics and Results

As mentioned in LaunchNotes (2023), quantitative metrics are numeric measures that help provide an objective comparison, and qualitative metrics are judgments that add context. In GeeksforGeeks (2025a), correctness, performance, and usability are mentioned among those parameters of software quality. Wells (2025) adds that metrics that sound good may not be useful if they do not actually provide value. Therefore, all the metrics presented below are linked to requirements provided in the Unit 3 design. According to Letaw (2024), a good requirement is a requirement that can be tested, and this is precisely why writing the non-functional requirements that way in Unit 3 made them measurable now.

Step 4 of the walkthrough measured accuracy, detection time, throughput, scoring time, memory, and response time (Figure 15). Accuracy and detection time were measured on 240 synthetic patients, which gave 224 consenting devices. The answer key records what each patient was built to be: 64 devices needed a call (56 that went quiet and 8 new devices that went quiet before they had a pattern) and 160 kept a normal rhythm. The system was compared with a fixed 48-hour rule, the kind of single threshold the project set out to replace. Table 5 and Figure 16 show the results. Precision is the share of correct flags, recall is the share of devices needing a call that were flagged, and the F1 score combines the two.

Table 5

Accuracy and Detection Time on 240 Synthetic Patients

Measure	Personal baseline	Fixed 48-hour rule
Precision	1.00	0.58
Recall	1.00	0.88

Measure	Personal baseline	Fixed 48-hour rule
F1 score	1.00	0.70
False alarms; missed devices	0; 0	40; 8
False-alarm rate, every-3-day senders	0%	100%
Hours to flag: twice daily, daily, every 3 days	25, 49, 142	48, 48, 48

Figure 15

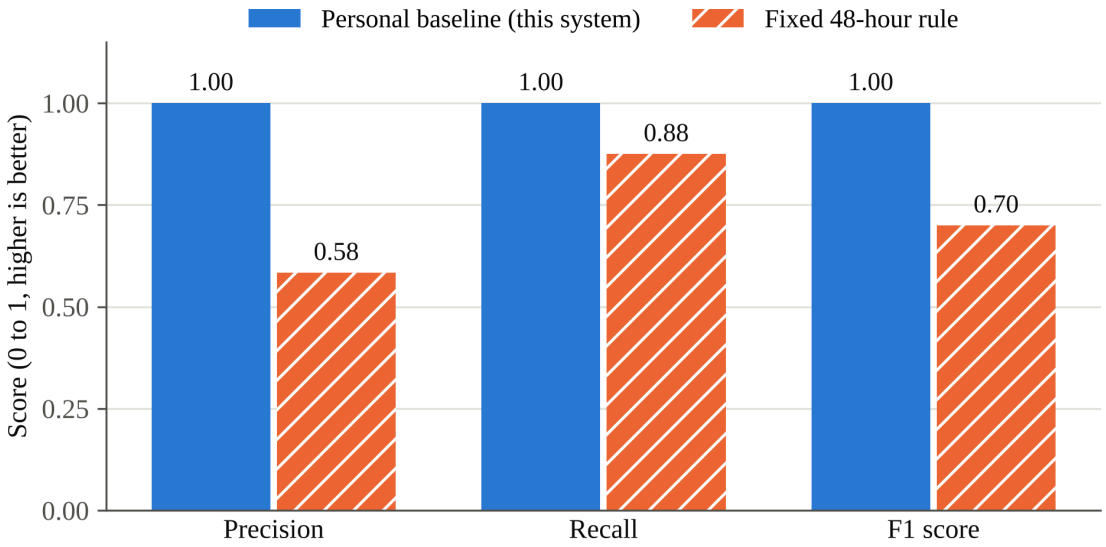
Evaluation Output

```
STEP 4  Evaluation
------------------
  Accuracy on 240 synthetic patients (224 devices)
    Rule                 Precision  Recall  False alarms  Missed
    Personal baseline         1.00    1.00             0       0
    Fixed 48-hour rule        0.58    0.88            40       8
    48-hour rule false alarms among normal daily senders: 0 of 120
    48-hour rule false alarms among normal every-3-day senders: 40 of 40
  Hours from last reading to flag (median, devices that went quiet)
    Twice a day      25 h   (fixed 48-hour rule: 48 h)
    Daily            49 h   (fixed 48-hour rule: 48 h)
    Every 3 days    142 h   (fixed 48-hour rule: 48 h)
  Speed with 4,800 patients (183,212 rows): ingestion 669,150 rows per second, scoring 4,480 devices in 160 ms
  Memory: the program peaked at 124 MB
  Queue API response time (100 requests): median 0.6 ms, 95th percentile 0.8 ms

(press Enter for the next step)
```

Figure 16

Precision, Recall, and F1 Score of the Two Rules



The personal baseline flagged all 64 devices that needed a call and raised no false alarms. The fixed rule raised 40 false alarms, every one among patients who normally send readings every three days, and missed all eight twice-daily devices that went quiet, because their silence reached only 28 to 34 hours. Detection time tells a more mixed story (Figure 17). The personal baseline flagged twice-daily devices after a median of 25 hours, about a day sooner than the fixed rule, and daily devices after 49 hours, about the same. For devices that normally send every three days, it needed a median of about 142 hours, roughly four days longer than the fixed rule.

Figure 17

Hours From Last Reading to Flag, by Sending Rhythm

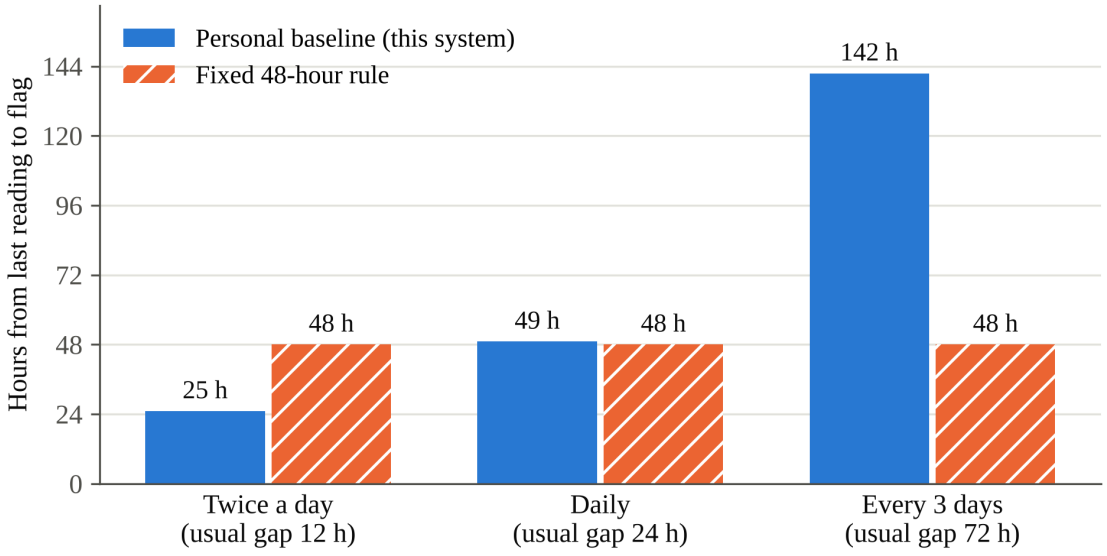


Table 6 shows how the system handled a population of 4,800 patients, twenty times the evaluation set. Ingestion processed about 669,000 rows per second, a scoring pass over all 4,480 devices took 160 milliseconds, and the whole program peaked at 124 MB of memory. The queue API answered 100 requests in a median of 0.6 milliseconds, with 95% answered within 0.8

milliseconds. The synthetic data flags about 29% of devices, a deliberately high rate for testing, so a real program would likely produce shorter queues than these tests.

Table 6

Load Test Results

Measure	Result
Vendor rows at 4,800 patients	183,212
Ingestion throughput	669,150 rows per second
Scoring pass, 4,480 devices	160 ms
Peak memory, whole program	124 MB
Queue API, 100 requests	Median 0.6 ms; 95% within 0.8 ms

Note. Measured on the development computer; times vary between computers and runs.

For the qualitative measure, I walked through a nurse's task on the dashboard: find who to call first, understand why, and record the result. Each step was judged against NFR-4, which requires the queue to show the reason for every rank, and against the general idea of usability as how user-friendly a system is (GeeksforGeeks, 2025a). Table 7 lists the seven checks. Five were met, and two were partly met.

Table 7

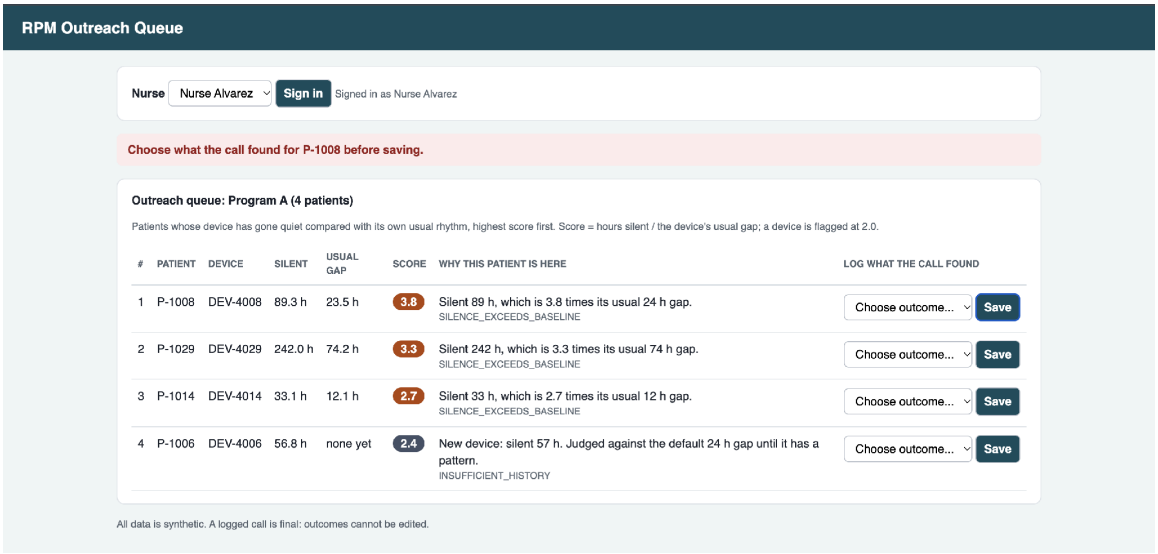
Interface Clarity Walkthrough

Check	Evidence	Rating
Who to call first is clear	Rank, score badge, highest first	Met
Why is clear before calling	Plain sentence beside each reason code	Met

Check	Evidence	Rating
New devices are explained	Says a default gap is used until a pattern exists	Met
Logging is quick	Choose an outcome, then Save	Met
Mistakes are caught	Saving without an outcome names the patient (Figure 17)	Met
Long silences are easy to read	Shown in hours, such as 242.0 h, not days	Partly met
The nurse can act at once	Phone number must be found elsewhere	Partly met

Figure 18

Dashboard Message for a Missing Outcome



DISCUSSION

This chapter discusses the findings in light of the research questions mentioned in previous chapters, summarizes the deployment approach, and points out the limitations and directions for future research.

5.1 Analyzing the Findings in Light of Prior Research

Chapter 2 established a gap in existing research. There are studies measuring RPM attrition after patients left (Su et al., 2025), with one of the arguments given by Athukorala (2026) being that if the program's workflow is optimized, it may last. While those studies have provided valuable information, they did not explain how a program can convert its own transmission logs into an early signal for reaching out to patients—one that is ranked and explained. Thus, these results can be seen as a first, synthetic test of one answer to that question, organized by the three research questions from Chapter 1.

When it comes to RQ1, which asked whether personal baselines reduce false flags compared with a fixed threshold, the synthetic results show that they do, lowering false alerts from 40 to zero. Besides the number of false alerts, their nature is also essential. All false alerts happen in the case of patients whose normal sending rhythm is slower than the threshold; hence, under a fixed rule, all such patients would be flagged every three days for behaving normally. According to Ancker et al. (2017), as workload, work complexity, and repeated alerts increase, alert fatigue grows and alerts are increasingly perceived as unimportant. When the same alerts repeat, front-line professionals will stop paying attention to them. It was also noted by Athukorala (2026) that technology anxiety or alert fatigue was reported in 9.3% of the RPM settings reviewed and digital health literacy gaps in 15%, with workflow friction often adding to both. The plain-sentence reasons serve the same purpose, since a nurse can check an alert before acting on it.

The second research question concerns the effect of the queue on detection time. Detection is important, as a silent device may take a long time to be discovered without a detection layer. The

results show that the system detects a quiet device after one to six days depending on rhythm. According to Su et al. (2025), 13% of 1,993 patients with diabetes invited to participate in the RPM program refused, and 16% withdrew, with younger patients and those in poor health being more likely to withdraw. Thus, earlier contact is important, especially for patients who are in poor condition. However, the comparison with the fixed rule shows a real trade-off, since the personal baseline is quicker for frequent senders but slower for those sending readings every three days, which means one of them may wait almost six days before getting flagged. This cost was not visible in the Unit 5 results, and it is the key finding of this evaluation. The third research question asked whether flag rates differ across cohorts, and under the fixed rule they differed sharply: 100% of normal every-three-day senders were flagged, while no daily senders were flagged. The personal baseline flagged neither group. This is important for the sake of fairness because slow senders can be people who have connectivity problems, for example, people living in remote areas. Treating their normal behavior as a problem would be unfair. Kirkland et al. (2023) placed the issues of RPM in the context of implementation based on delivery strategies, barriers, and facilitators rather than focusing on technology. From this perspective, detection rules are part of the program's workflow and should be checked with the staff who use them. NFR-7 requires cohort disparities to be measurable from the call log. The log now records every outcome; however, the cohort report has not been built yet, so the requirement is partly met.

The performance results address the remaining requirements. NFR-3 states that an entire scoring pass must be completed within the scheduled interval, and in testing, 4,800 patients took less than half a second against a planned hourly schedule. NFR-6 states that scoring must run independently of the dashboard application in order for the latter not to be slowed down by the

process of scoring. The design meets this demand in structure, because the dashboard only reads saved scores. Currently, scoring is done just once at the launch of the software, and future development of separate scoring will happen in the form of scheduled processes. Michaud et al. (2018) studied the results of implementation of RPM in the context of providing patient care for people suffering from type 2 diabetes after being discharged from the hospital, part of the evidence that keeping patients engaged in RPM is worth the effort. A detection layer that stays fast as a program grows protects that value. Table 8 summarizes how each non-functional requirement stands after Unit 7.

Table 8

Non-Functional Requirements Against the Evidence

Requirement	**Evidence**	**Status**
NFR-1 Encryption in transit and at rest	TLS planned at the HTTPS proxy; the prototype keeps data in memory, not yet on an encrypted disk	Partly met
NFR-2 Server-side authorization	Tokens checked on the server; 401 and 403 integration checks	Met
NFR-3 Scoring within the interval	160 ms for 4,480 devices against an hourly schedule	Met
NFR-4 Reason shown for each rank	Plain sentence and code on every row	Met
NFR-5 Repeatable ingestion	A repeated reading is stored once	Met

Requirement	Evidence	Status
NFR-6 Scoring independent of the dashboard	Dashboard reads saved scores; scheduled rescoring not built	Partly met
NFR-7 Cohort disparities measurable	Measured on synthetic data; call-log report not built	Partly met

5.2 Deployment Approach and Configuration Summary

Sumo Logic (n.d.) describes deployment as making software available to the end user once all necessary measures have been taken to prepare and test it. These preparations include getting the official Python container image and configuring environment variables on the intended host. Thereafter, unit tests and integration checks of the walkthrough are performed on a testing server. The final step in this phase is switching traffic to the program and monitoring the health check and the first scoring pass in a live environment. Three options were analyzed. A managed platform is easily scalable but would place patient data on cloud servers outside the organization's network. On the other hand, a server installed on-site does not require data sharing but has to be set up manually each time. Finally, the use of a Docker image allows running the program on the same version of Python, ensuring that the program will behave similarly on all computers. The selected option is running an official Python container on the organization server behind an existing HTTPS proxy. Blue-green deployment (ByteByteGo, 2023) means starting the new container alongside the old one, routing traffic only after the health check is passed, and the ability to revert traffic if something goes wrong. A canary deployment would become useful once several programs share the system. After launching the program, it is important to measure the frequency of deployment, change failure rate, and mean time to recovery as advised by Wells (2025).

The configuration management concept unites all of these steps. As stated by Summers (2020), the primary goals of configuration management are consistency, controlled change, and integrity of the product, while Bennett (2024) describes its components as identification of configuration, establishing a baseline, change control, status accounting, and auditing. In this project, every tagged release represents a baseline, while its release notes are the status record. All of this is important because, according to Davies (2024), flexibility of activities performed by the team and accountability are achieved via commit history, comparing new and previous versions of code and reverting to the previous one. All runtime settings are environment variables listed at the top of the program, so the same file runs in every environment without code changes.

The deployment plan completes the idea of CI/CD presented in Unit 4. Narasimhan et al. (2025) highlight quick deployments and higher quality of software products as the main advantages of DevOps, while Red Hat (2025) discusses security checks. The next stage of the project is to create a workflow running unit tests and a walkthrough for every pull request, and release new versions only if these tests pass.

5.3 Limitations

The most important limitation is the data. The synthetic patients were built with clearly separated rhythms and silences, so perfect precision and recall show that the rule works as designed, not that it will be as accurate on real transmission logs. Real devices fail in messier ways: batteries die slowly, patients travel, and vendors delay uploads. The second limitation is that usability was judged by me rather than by nurses. My walkthrough found real problems, but it cannot show whether nurses trust the queue or finish their work faster. Third, all timings come from one computer and a single-process server, so they show scale and trends rather than production

figures. Fourth, the sign-in is a demonstration stand-in. A real deployment would use the organization's identity system with passwords and expiring sessions. Fifth, the prototype keeps its database in memory, so call outcomes are lost when it stops; a production version needs a database file on an encrypted disk. Sixth, the Docker deployment and the blue-green release are planned but have not been run on a server. Finally, detection is slow for patients who send every three days, and memory grows with the size of the input file because the whole file is read at once. SQLite suits one server and one program well, but several application servers sharing the data would need a database server. New devices are judged against a 24-hour default until they have five gaps, which suits daily devices but could flag a device prescribed for weekly readings too early.

5.4 Recommendations for Future Research

The first step suggested is to examine the proposed theory (with ethics approval and a privacy assessment) on real de-identified transmission logs. Parzych (2019) mentions that utilizing live data carries the danger of leaked or destroyed data, therefore, an analysis should be performed on an internal copy. Secondly, a usability test should be carried out with monitoring nurses that will measure how long it takes to identify the first patient, measure mistakes, and understand the level of belief in the decisions made. The third step is to perform a test of the whole threshold, i.e. change its values from 1.5 to 3 on real data and examine a cap that flags devices that have been silent for over four days, hence enabling less time for sparsely sending devices to be detected. The following action to be conducted is comparison of the rate of provided flags and confirmed issue rates among the groups, for instance, rural and urban patients, through actual calls. The fifth recommendation is to analyze whether earlier calls prevent termination of the program, the challenge researched by Su et al. (2025). Finally, the last step would be to establish

time-of-the-day trends that will allow for eliminating the noise because some patients only take measurements in the morning so a silence that starts in the evening means less than one that starts at breakfast.

CONCLUSION AND FUTURE WORK

6.1 Summary of Main Findings and Implications

This project was designed to identify silent RPM devices promptly without causing too much strain on the nurses. The results produced from Units 4 to 7 consisted of a working, tested, and integrated system using scoring for each device in accordance with its rhythm, describing each flag in an easy-to-understand format, and directing all patients to the right nurse. In 240 simulated patients, all devices that needed a call were spotted without any false alarms, while a fixed 48-hour rule raised 40 false alarms and missed eight devices. All the false alarms triggered by the fixed rule were from patients sending readings every third day, suggesting that a single threshold causes trouble for certain groups of patients and the medical staff. The finished application passed all seven integration tests and 21 unit tests and enforced program boundaries on the server. The system also scored 4,800 patients in under half a second per scoring pass. The main implication is that a monitoring program can turn the data it already collects into a short and clear queue, which must be investigated in practice. The evaluation provided evidence that there is a drawback in personal baselines since they are slower to flag patients who send readings infrequently; in turn, a production version needs a cap for them.

6.2 Contributions to the Field

The project can be considered important for three reasons. First, it transitions from the fixed threshold system to a per-device baseline that explains itself, which responds to the alert fatigue

described by Ancker et al. (2017) and also to the workflow issue reviewed by Athukorala (2026). Second, it provides the possibility to assess the threshold on the basis of such criteria as a synthetic answer key, a comparison rule, the time of detection, and a cohort check, instead of relying on accuracy only. Finally, it provides a small but reproducible design of the integrated application with authorization on the server, an append-only log of calls, and documented configuration.

6.3 Lessons Learned

The most important lesson to be learned during this process is that testing results influence the design, instead of just confirming it. The scoring rule became testable only after it was separated from the database and the clock, with the first testing revealing four bugs concealed by Unit 4's demonstration. One more lesson is that one index of accuracy can conceal the trade-off involved. The detection-time measure highlighted a weakness that was not revealed by precision and recall. A third lesson is that a simpler system is easier to trust. Keeping everything in one file with one command walking through all the steps of the process made the testing, demonstration, and explanation of the system much easier.

6.4 Challenges

When planning occurred, the first hurdle was the base of literature. Initial materials discussed research method instead of problem sphere, leaving groundless thesis of the project related to the latency in detection. This could only be corrected by searching the literature on remote patient monitoring directly, and the search led to attrition data mentioned in Section 2.1. The search changed the general approach to the project: the result which indicated that withdrawals happen

mostly among patients with worse initial health changed the focus from the efficiency of early detection to its safety indicators.

The second obstacle refers to decision-making during designing of the system. Initially it was decided that the scoring engine will be built on the threshold for the whole program which seems to be easier to launch and operate. However, the term alert fatigue showed that this threshold is not going to work with those patients, whose transmission pattern varies, thus producing noise for those who are least able to cope with it. The option of having a rolling baseline for every patient was chosen instead, which added complexity but was the only reasonable choice available.

The third challenge was discipline in terms of scope. Predictive modeling in clinical deterioration sounds appealing but in practice it requires working with real patients' data and conducting clinical validation of the system which is impossible within one term. Sticking to exclusion criteria is an important requirement which has to be included into the plan explicitly.

A fourth challenge was practical rather than conceptual. Producing the architecture diagram took several iterations before the layout was legible, with early versions suffering from overlapping labels and edge routing that obscured the data flow. This is a minor issue but a real one, since a diagram that cannot be read at page size communicates nothing.

During development, the first challenge was refactoring the Unit 4 code without changing results the instructor had already seen. Comparing outputs line by line after each change solved this. The second was version control. Pushing directly to GitHub failed because of permission settings, so each release was uploaded and merged through the GitHub website, with branches and tags kept in order by hand. The third came during integration. The web server and the

walkthrough share one database, so a lock and one transaction per scoring pass were added to keep them from interfering. Demonstration data also had to be generated at start-up so that silences were measured from the current time. Finally, the first integrated version was split across many files, which made it hard to run and explain, so the final version keeps everything in one file, as in Units 4 and 5.

6.5 Next Improvements

The next version should add a cap so that no device is silent for more than four days without a flag, show long silences in days, and add a link to the patient's record so the nurse can call at once. On the technical side, the scorer should run as its own process on a schedule, the database should move to a file on an encrypted disk, the vendor file should be read in a stream rather than all at once, and the cohort report required by NFR-7 should be built from the call log. A GitHub Actions pipeline should run the tests on every pull request. Most importantly, the dashboard should be reviewed with monitoring nurses, in a formal study, so that the next version answers their needs and not only the developer's.

REFERENCES

Amazon Web Services. (n.d.). *What is unit testing?* https://aws.amazon.com/what-is/unit-testing/

Ancker, J. S., Edwards, A., Nosal, S., Hauser, D., Mauer, E., & Kaushal, R. (2017). Effects of workload, work complexity, and repeated alerts on alert fatigue in a clinical decision support system. *BMC Medical Informatics and Decision Making, 17*(1), Article 36. https://doi.org/10.1186/s12911-017-0430-8

Athukorala, S. C. (2026). Clinical efficacy and patient adherence hurdles of remote patient monitoring (RPM) frameworks in managing hypertension and type 2 diabetes: A scoping review. *Cureus*. https://doi.org/10.7759/cureus.113642

Bennett, L. (2024, August 13). *Software configuration management in software engineering*. Guru99. https://www.guru99.com/software-configuration-management-tutorial.html

Bergmann, F. (2021, March 17). *My favorite open source project management tools*. Opensource.com. https://opensource.com/article/21/3/open-source-project-management

BrowserStack. (2025, May 29). *What is system testing? (Examples, use cases, types)*. https://www.browserstack.com/guide/what-is-system-testing

ByteByteGo. (2023, June 6). *Top 5 most-used deployment strategies* [Video]. YouTube. https://www.youtube.com/watch?v=AWVTKBUnoIg

Davies, S. (2024). *Blueprints: Creating, describing, and implementing designs for larger-scale software projects* (Version 2.5). University of Mary Washington. https://open.umn.edu/opentextbooks/textbooks/blueprints-creating-describing-and-implementing-designs-for-larger-scale-software-projects-davis

Editorial Team. (n.d.). *How to create a software design document: Steps & template*. Bit.ai Blog. https://blog.bit.ai/software-design-document/

GeeksforGeeks. (2025a, July 15). *Measuring software quality using quality metrics*. https://www.geeksforgeeks.org/software-engineering/measuring-software-quality-using-quality-metrics/

GeeksforGeeks. (2025b, July 22). *System testing – Software testing*. https://www.geeksforgeeks.org/software-testing/system-testing/

GeeksforGeeks. (2025c, July 22). *Unit testing – Software testing*. https://www.geeksforgeeks.org/software-testing/unit-testing-software-testing/

Hrupa, A. (2024, March 19). *What is system testing in software testing*. Testfort. https://testfort.com/blog/what-is-system-testing-in-software-testing

Kirkland, E. B., Johnson, E., Bays, C., Marsden, J., Verdin, R., Ford, D., King, K., & Sterba, K. R. (2023). Diabetes remote monitoring program implementation: A mixed methods analysis of delivery strategies, barriers and facilitators. *Telemedicine Reports*. https://doi.org/10.1089/tmr.2022.0038

Krüger, G. (2025, July 11). *How to write a software requirements specification (SRS) document*. Perforce. https://www.perforce.com/blog/alm/how-write-software-requirements-specification-srs-document

LaunchNotes. (2023, September 1). *Qualitative vs quantitative metrics: A comprehensive comparison*. https://www.launchnotes.com/blog/qualitative-vs-quantitative-metrics-a-comprehensive-comparison

Letaw, L. (2024). *Handbook of software engineering methods* [Open textbook]. Oregon State University. https://open.oregonstate.education/setextbook/

Lucidchart. (n.d.). *How to create software design documents*. https://www.lucidchart.com/blog/how-to-create-software-design-documents

Michaud, T. L., Siahpush, M., Schwab, R. J., Eiland, L. A., DeVany, M., Hansen, G., Slachetka, T. S., Boilesen, E., Tak, H. J., Wilson, F. A., Wang, H., Pagán, J. A., & Su, D. (2018). Remote patient monitoring and clinical outcomes for postdischarge patients with type 2 diabetes. *Population Health Management*. https://doi.org/10.1089/pop.2017.0175

Miller, R. (2024, February 28). *Capstone project outline essentials: A step-by-step guide to success*. DoMyEssay. https://domyessay.com/blog/capstone-project-outline

Narasimhan, A., Sham, R., & Subramanian, H. (2025, October 15). *A beginner's handbook to DevOps* [White paper]. Dell Technologies. https://awsprod.merlot.org/merlot/viewMaterial.htm?id=773474552

Oguz, A. (2025). *Project management* (2nd ed.). MSL Academic Endeavors. https://pressbooks.ulib.csuohio.edu/projectmanagement2ndedition/

Paperpile LLC. (n.d.). *How to write a problem statement*. https://paperpile.com/g/how-to-write-a-problem-statement/

Parzych, D. (2019, December 30). *8 must-read DevOps articles for success in 2020*. Opensource.com. https://opensource.com/article/19/12/devops-resources

PhD Assistance. (2020, December 14). *How to identify the appropriate PhD dissertation research problem*. https://www.phdassistance.com/blog/how-to-identify-the-appropriate-phd-dissertation-research-problem-identification-in-computer-science-research-explain-the-ways-to-evaluate-it/

Rana, K. (2023, September 23). *What is a test case? Test case examples*. ArtOfTesting. https://artoftesting.com/test-case

Red Hat. (2025, April 29). *What is DevOps?* https://www.redhat.com/en/topics/devops/what-is-devops

Sheppard, V. A. (2019). *An introduction to research methods in sociology*. Justice Institute of British Columbia. https://pressbooks.bccampus.ca/researchmethods/

Su, D., Michaud, T. L., Ern, J., Li, J., Chen, L., Li, Y., Shi, L., Zhang, D., Andersen, J., & Pagán, J. A. (2025). Diabetes management through remote patient monitoring: A mixed-methods evaluation of program enrollment and attrition. *Healthcare, 13*(7), Article 698. https://doi.org/10.3390/healthcare13070698

Summers, B. L. (2020). *Effective methods for software engineering*. Auerbach Publishers.

Sumo Logic. (n.d.). *What is software deployment?* https://www.sumologic.com/glossary/software-development/

Walker, V. (2022, March 16). *14 software architecture design patterns to know*. Red Hat. https://www.redhat.com/en/blog/14-software-architecture-patterns

Wells, J. (2025, April 16). *Top 12 software quality metrics to measure and why*. LinearB. https://linearb.io/blog/software-quality-metrics

APPENDIX A: CODE SAMPLES

Figure A1 shows the Unit 7 fix for reading times with a time zone (defect D3), and Figure A2 shows the two core scoring functions in version 0.4.0.

Figure A1

Fix for Defect D3 in clean_time()

```
def clean_time(text, as_of):
    """Return the reading time in one standard format, or None if it is
    missing, unreadable, or later than as_of."""
    try:
        when = datetime.fromisoformat(text)
        if when.tzinfo is not None:                  # e.g. ...Z or ...+00:00
            when = when.astimezone().replace(tzinfo=None)   # to local time (Unit 7 fix)
        if when > as_of:
            return None
        return when.isoformat(timespec="seconds")
    except (TypeError, ValueError):
        return None
```

Figure A2

Core Scoring Functions usual_gap() and score_device() (Version 0.4.0)

```python
def usual_gap(times):
    """Median hours between sessions over the last 30 days. Readings less
    than an hour apart count as one session. None if there is no pattern yet."""
    times = sorted(times)
    sessions = [t for i, t in enumerate(times)
                if i == 0 or t - times[i - 1] > timedelta(minutes=SESSION_MINUTES)]
    recent = [t for t in sessions if t >= sessions[-1] - timedelta(days=BASELINE_DAYS)]
    gaps = [(b - a).total_seconds() / 3600 for a, b in zip(recent, recent[1:])]
    if len(gaps) < MIN_GAPS:
        return None
    return statistics.median(gaps)


def score_device(times, as_of, enrolled_on):
    """How many usual gaps the device has been silent for, as of a given time."""
    times = [t for t in times if t <= as_of]          # ignore future readings
    last = max(times) if times else enrolled_on       # never sent? count from enrollment
    silent = (as_of - last).total_seconds() / 3600
    usual = usual_gap(times)
    score = silent / (usual or DEFAULT_GAP_HOURS)
    flagged = score >= FLAG_AT and silent >= MIN_SILENT_HOURS
    if usual is None:
        reason = "INSUFFICIENT_HISTORY"
    elif flagged:
        reason = "SILENCE_EXCEEDS_BASELINE"
    else:
        reason = "WITHIN_PERSONAL_PATTERN"
    return {"silent": silent, "usual": usual, "score": score,
            "reason": reason, "flagged": flagged}
```

APPENDIX B: DATA SAMPLES AND SETTINGS

Table B1 shows sample rows from the synthetic vendor file, including the two deliberate bad rows at the end. Table B2 lists the settings and their defaults.

Table B1

Sample Rows From the Synthetic Vendor File

patient_id	device_id	reading_time	systolic	pulse
P-1001	DEV-4001	2026-10-21T02:45:22	121	89
P-1001	DEV-4001	2026-10-20T02:18:53	116	93
P-1001	DEV-4001	2026-10-18T22:12:13	123	68
P-1001	DEV-4001	10/21/2026	150	56
P-1007	DEV-4007	2026-10-23T17:55:40	122	72

Table B2

Settings and Their Defaults

Setting	Default	Purpose
RPM_HOST (environment)	127.0.0.1	Address to listen on
RPM_PORT (environment)	8000	Server port
RPM_FLAG_AT (environment)	2.0	Usual gaps of silence before a flag
MIN_SILENT_HOURS (in code)	24	Shortest silence that can be flagged
BASELINE_DAYS (in code)	30	Days used to learn the rhythm
SESSION_MINUTES (in code)	60	Readings this close together are one session

APPENDIX C: ADDITIONAL SCREENSHOTS

Figure C1 shows the dashboard after a call is logged in version 0.4.0.

Figure C1

Dashboard After a Call Is Logged

